

PARTE 1 · FUNDAMENTOS ABSOLUTOS

De cero absoluto a nivel experto



CLIENTES

id	nombre	email
1	Ana	ana@...
2	Luis	luis@...

```
SELECT nombre
FROM clientes
WHERE id = 1;

-- Tu 1ª consulta
```

BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Contenido de esta parte

Antes de empezar

1.1 Dato frente a información

1.2 Por qué existen las bases de datos

1.3 Qué es una base de datos y qué es un gestor

1.4 Anatomía de una tabla

1.5 Tipos de datos

1.6 Valores, nulos e identificadores

1.7 Claves: primaria y extranjera

1.8 Índices

1.9 Consultas

1.10 El entorno: cliente, servidor, API, VPS y nube

Resumen de la Parte 1

Glosario de la Parte 1

Mini examen de la Parte 1

Parte 1. Fundamentos absolutos

Antes de empezar

Esta parte no necesita ningún conocimiento previo. No hace falta saber programar, ni haber usado nunca una base de datos, ni conocer ninguna palabra técnica. Todo lo que aparezca se explica la primera vez que se usa, y todo lo que se explica se comprueba con un ejemplo que funciona de verdad.

Al terminarla serás capaz de:

- explicar con tus palabras qué es un dato, una base de datos y un gestor de bases de datos;
- crear una tabla, insertar filas y leerlas en PostgreSQL;
- leer una tabla y decir qué es cada una de sus partes (tabla, fila, columna, registro, campo y valor);
- elegir el tipo de dato adecuado para cada columna;
- entender qué es NULL, una clave primaria, una clave extranjera, un índice y una consulta, comprobando cómo se comporta cada uno;
- seguir el recorrido de un dato desde la pantalla de una aplicación hasta el disco donde se guarda.

Cómo se organiza cada punto

Cada punto del manual sigue los mismos pasos, siempre en el mismo orden:

1. **¿Qué es?, ¿Para qué sirve?, ¿Por qué lo necesito? y ¿Cómo funciona?:** la teoría, con palabras sencillas.
2. **Paso a paso:** lo que debes hacer, en orden, y lo que deberías ver en cada paso.
3. **Código y resultado:** el código completo, escrito en texto para que lo copies, seguido de **la captura real de su ejecución** y una explicación de lo que muestra, línea a línea.
4. **Ejemplo sencillo y ejemplo real.**
5. **Profundizando:** variantes, casos límite y detalles que usan los profesionales.
6. **Ejercicio, solución, error habitual, buena práctica y comprobación.**

Cómo se ejecutan los ejemplos

Todo el código de esta parte se ha ejecutado en **PostgreSQL 16.14**. Las capturas con fondo oscuro son la **salida real de `psql`**, el programa de terminal con el que se habla con PostgreSQL. Los mensajes están en inglés porque así los emite el servidor. Las figuras que son simples ilustraciones lo dicen en su texto.

Para ejecutar tú los ejemplos necesitas PostgreSQL y `psql`. La **Parte 0** te enseña, paso a paso, a instalarlos, a abrir `psql`, dónde escribir el código y cómo guardarlo en archivos. **Si todavía no la**

has hecho, hazla antes de seguir. Puedes leer esta parte sin ejecutar nada y volver más tarde a practicar. Cada captura muestra en su prompt en qué base de datos se ejecuta (`ensayo`, `tienda`, `crm` ...): son las que creaste o crearás con `CREATE DATABASE` (punto 0.5). Para recordar cómo se lee una sesión de `psql`:

Lo que ves	Qué significa
<code>tienda=#</code>	El prompt : <code>psql</code> está conectado a la base de datos <code>tienda</code> y espera que escribas una orden
<code>tienda-#</code>	La orden anterior no ha terminado (le falta el <code>;</code>) y continúas escribiéndola en otra línea
<code>;</code> al final	Marca el final de una sentencia SQL. Sin él, <code>psql</code> sigue esperando
<code>\d</code> , <code>\dt</code> , <code>\l</code>	Comandos propios de <code>psql</code> (empiezan por barra invertida). No llevan <code>;</code>
<code>CREATE TABLE</code> , <code>INSERT 0 3</code> , <code>UPDATE 1</code>	La respuesta del gestor: qué hizo y a cuántas filas afectó
<code>ERROR:</code>	La orden no se ejecutó . La línea siguiente (<code>DETAIL:</code>) explica por qué
<code>(3 rows)</code>	El resultado de una consulta tenía 3 filas
<code>t</code> y <code>f</code>	<code>true</code> (verdadero) y <code>false</code> (falso)
Celda vacía	El valor es NULL: no hay valor (punto 1.6)
<code>\q</code>	Sale de <code>psql</code>

No hace falta entender todavía cada palabra de SQL (el lenguaje de las bases de datos): se explica con calma en la Parte 5. En esta parte basta con leer cada orden con la explicación que la acompaña.

Los datos de ejemplo de esta parte

Usaremos una pequeña tienda online con clientes y pedidos. Los clientes son Ana García, Luis Pérez y Marta Ruiz. Ana hizo dos pedidos, Marta uno y Luis ninguno. Volveremos a ellos en cada punto.

1.1 Dato frente a información

¿Qué es?

Un **dato** es un valor suelto, sin contexto. Por ejemplo: «Ana», «34», «Madrid», «45.90», «2026-03-02». Si ves uno de ellos escrito en un papel, no sabes qué significa.

La **información** es lo que obtienes cuando los datos se colocan en un contexto que les da sentido. «Ana tiene 34 años y vive en Madrid» es información: sabes de quién se habla, qué es el 34 y qué es Madrid.

La diferencia es la misma que entre ladrillos y una pared. Los ladrillos (datos) no sirven de nada amontonados; colocados con orden y con un plano (el contexto) forman una pared (información).

¿Para qué sirve distinguirlos?

Para decidir qué guardar y cómo. Una base de datos no guarda «información» como tal: guarda datos con la estructura suficiente para que, al pedirlos, devuelvan información. Esa estructura es lo que veremos en todo el manual.

¿Por qué lo necesito?

Imagina que alguien te da una lista con «38», «38», «37», «39». ¿Son edades? ¿Euros? ¿Grados de fiebre? ¿Tallas de zapato? Sin contexto, la lista no sirve. Si una aplicación guarda ese «38» sin decir qué significa, dentro de unos meses nadie sabrá interpretarlo, ni siquiera quien la programó.

Cada vez que diseñes una base de datos te harás las mismas preguntas sobre cada dato: ¿de qué cosa es? ¿a quién pertenece? ¿en qué unidad está? ¿cuándo se midió?

¿Cómo funciona?

El contexto de un dato se compone de pistas que se combinan:

- **El nombre de la columna:** «edad», «precio» o «ciudad» dicen qué es el dato.
- **La fila a la que pertenece:** dice a quién o a qué se refiere (Ana, el pedido 101).
- **El tipo de dato:** dice si es un número, un texto o una fecha (punto 1.5).
- **La unidad,** cuando hace falta: euros, kilos, años. A menudo va en el nombre de la columna (`precio_eur`).

Cuando las cuatro pistas están, el dato se convierte en información.

Paso a paso: de datos sueltos a información

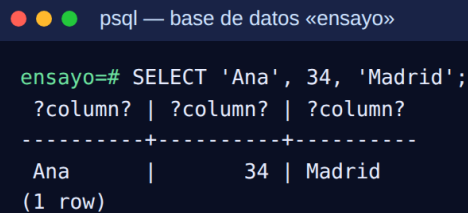
1. **Pide los datos sin ninguna estructura** y observa que no dicen nada.
2. **Decide de qué cosa hablas:** una persona.
3. **Crea una columna por cada característica:** `nombre`, `edad` y `ciudad`, cada una con su tipo.
4. **Guarda los datos en una fila,** cada uno bajo su columna.

5. **Pide que el gestor construya una frase** con ellos: así compruebas que ya hay información.

Código y resultado

Primero, tres datos sueltos. La orden `SELECT` («selecciona») devuelve lo que le pides, aunque no esté guardado en ninguna tabla:

```
SELECT 'Ana', 34, 'Madrid';
```



```
psql — base de datos «ensayo»
ensayo=# SELECT 'Ana', 34, 'Madrid';
?column? | ?column? | ?column?
-----+-----+-----
Ana      |      34  | Madrid
(1 row)
```

Figura 1.1-a. Captura real: tres datos sueltos. PostgreSQL no sabe qué son.

Qué ves en la imagen. El resultado tiene tres columnas que se llaman `?column?`: es la forma de PostgreSQL de decir «no tengo nombre para esto». Ves `Ana`, `34` y `Madrid`, pero nada indica que sean un nombre, una edad y una ciudad. Son datos.

Ahora los guardamos con estructura:

```
CREATE TABLE personas (
  nombre text,
  edad integer,
  ciudad text
);

INSERT INTO personas VALUES ('Ana', 34, 'Madrid');

SELECT * FROM personas;

SELECT nombre || ' tiene ' || edad || ' años y vive en ' || ciudad AS
informacion
FROM personas;
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE personas (
ensayo-#     nombre text,
ensayo-#     edad  integer,
ensayo-#     ciudad text
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO personas VALUES ('Ana', 34, 'Madrid');
INSERT 0 1

ensayo=# SELECT * FROM personas;
 nombre | edad | ciudad
-----+-----+-----
  Ana   |   34 | Madrid
(1 row)

ensayo=# SELECT nombre || ' tiene ' || edad || ' años y vive en ' || ciudad AS informacion
ensayo-# FROM personas;
      informacion
-----
Ana tiene 34 años y vive en Madrid
(1 row)

```

Figura 1.1-b. Captura real: los mismos datos con contexto se convierten en información.

Qué ves en la imagen, orden por orden.

1. `CREATE TABLE personas (...)` crea una tabla con tres columnas. Cada línea entre paréntesis es una columna con su nombre y su tipo: `nombre text` (texto), `edad integer` (número entero) y `ciudad text`. La respuesta `CREATE TABLE` confirma que existe.
2. `INSERT INTO personas VALUES ('Ana', 34, 'Madrid')` guarda una fila. Los valores se escriben en el mismo orden que las columnas. Los textos van entre comillas simples; los números, sin ellas. La respuesta `INSERT 0 1` significa «se ha insertado 1 fila».
3. `SELECT FROM personas` pide todas las columnas (significa «todas») de la tabla. Ahora cada dato tiene su nombre de columna encima: ya sabes cuál es el nombre, cuál la edad y cuál la ciudad.
4. La última consulta construye una frase uniendo los datos con `||` (que significa «pegar»). El resultado, en la columna `informacion`, es *Ana tiene 34 años y vive en Madrid*. El gestor ha pasado de datos a información gracias a la estructura.



Figura 1.1-c. Ilustración: de datos sueltos a información.

Qué ves en la imagen. A la izquierda, tres datos sueltos en cajas. A la derecha, esos mismos datos bajo las columnas `nombre`, `edad` y `ciudad`. No ha cambiado ningún dato: solo ha cambiado el contexto.

Ejemplo sencillo

Tres datos: «Luis», «612345678», «luis@ejemplo.com». Con las columnas `nombre`, `telefono` y `email` forman una fila que dice «Luis puede ser contactado por teléfono en el 612345678 o por correo en luis@ejemplo.com».

Ejemplo real

Una tienda online guarda «101», «2026-03-02» y «45.90». Sin estructura son tres números sin sentido. Con las columnas `id_pedido`, `fecha` y `total_eur`, el sistema sabe que el pedido 101 se hizo el 2 de marzo de 2026 y costó 45,90 euros. Con eso puede mostrar el pedido, sumar las ventas del mes o enviar la factura.

Profundizando

El significado vive en el diseño, no en el dato. Cambiar el nombre de una columna cambia el significado de todos sus datos. Por eso el nombre importa tanto como el valor.

La unidad es parte del contexto. Si una columna mezcla euros y dólares, los datos individuales son correctos y la información es falsa. Pon la unidad en el nombre (`precio_eur`, `peso_kg`) o guarda la moneda en otra columna.

Dato derivado frente a dato guardado. La frase de la figura 1.1-b no se guarda: se **calcula** cada vez que se pide. Guardar solo los datos básicos y calcular lo demás evita contradicciones (por ejemplo, guardar la edad y la fecha de nacimiento a la vez).

Ejercicio

1. Con los datos «Luis», «612345678» y «luis@ejemplo.com», escribe el `CREATE TABLE` de una tabla `contactos` con tres columnas y su tipo, y el `INSERT` que guarda la fila.
2. Te dan el dato «19.95». Escribe tres significados posibles y di qué información adicional necesitarías para saber cuál es el correcto.
3. Una columna se llama `dato1`. ¿Por qué es mal nombre? ¿Cómo la llamarías si guarda el precio de un producto en euros?

Solución

1. La tabla y la fila:

```
CREATE TABLE contactos (
  nombre text,
  telefono text,
  email text
);
INSERT INTO contactos VALUES ('Luis', '612345678', 'luis@ejemplo.com');
```

El teléfono se guarda como texto, no como número (lo razonamos en el punto 1.5).

1. Podría ser un precio en euros, una altura en metros o unos grados de temperatura. Necesitas el nombre de la columna, la fila a la que pertenece y la unidad.
2. `dato1` no dice qué guarda; en seis meses nadie lo recordará. Un buen nombre es `precio_eur`.

Error habitual

Guardar datos con nombres que no explican nada (`dato1`, `campo_x`, `valor`) o mezclar unidades en la misma columna.

```
-- Mal: nadie sabe qué significa cada columna
CREATE TABLE medidas (dato1 numeric, dato2 numeric);
-- Bien: nombre y unidad en cada columna
CREATE TABLE medidas_ok (temperatura_c numeric, humedad_pct numeric);
```

Buena práctica

Dale a cada columna un nombre que explique su significado y, si hace falta, su unidad. Escríbelo pensando en alguien que no conoce tu proyecto.

Comprobación

Elige cualquier dato de una aplicación que uses y responde: ¿de qué cosa es? ¿a quién pertenece? ¿en qué unidad está? Después comprueba en `psql` que una consulta devuelve información legible (con nombres de columna), no una fila de valores anónimos.

1.2 Por qué existen las bases de datos

¿Qué es?

Una base de datos es la solución a un problema muy antiguo: guardar muchos datos sin que se desordenen, se repitan, se pierdan o se contradigan. Antes de que existieran, la gente usaba cuadernos, fichas de papel, archivos sueltos en el ordenador y, más tarde, hojas de cálculo.

¿Para qué sirve?

Para cuatro cosas principales:

- **Guardar** datos de forma fiable, de modo que no se pierdan si se apaga el ordenador.
- **Encontrar** un dato concreto entre millones, en una fracción de segundo.
- **Evitar duplicados y contradicciones**, guardando cada dato una sola vez.
- **Permitir el uso simultáneo**: muchas personas y programas pueden leer y modificar los datos a la vez sin pisarse unos a otros.

¿Por qué lo necesito?

Una hoja de cálculo funciona bien con pocos datos y una sola persona. Cuando el proyecto crece (miles de clientes, varios empleados, una web y una app que leen y escriben a la vez), empieza a fallar:

- Dos personas editan el mismo archivo y una pisa los cambios de la otra.
- El mismo cliente está escrito en varias filas, y cada una dice una cosa distinta.
- Nada impide escribir «hola» en una celda de precio.
- Cualquiera que abra el archivo ve todos los datos; no se puede dar permiso solo para una parte.
- Cuando el archivo es muy grande, abrirlo y buscar en él se vuelve lentísimo.

Una base de datos está diseñada para resolver precisamente estos problemas.

¿Cómo funciona?

Impone reglas que la hoja de cálculo no impone:

1. **Cada dato tiene un tipo.** Una columna de importes solo admite números.
2. **Cada fila se identifica de forma única.** No pueden existir dos filas indistinguibles.
3. **Los datos relacionados se guardan una vez y se enlazan.** El pedido no copia los datos del cliente: apunta a él.
4. **Hay control de acceso.** Cada usuario o programa puede tener permiso solo para lo que necesita.
5. **Los cambios son seguros.** Si algo falla a mitad de una operación, se deshace entera en lugar de dejar los datos a medias.

Paso a paso: reproducir el problema de la hoja de cálculo y resolverlo

1. **Crea una tabla sin ninguna regla**, que se comporta como una hoja de cálculo: cualquier texto en cualquier columna.
2. **Introduce los datos de un mismo cliente en tres filas**, con un pequeño descuido en una de ellas.
3. **Pregunta por los clientes distintos** y observa la contradicción.
4. **Rediseña con dos tablas** (clientes y pedidos), de modo que el cliente exista una sola vez.
5. **Cambia el teléfono en un único sitio** y comprueba que todos los pedidos lo reflejan.
6. **Intenta guardar un dato absurdo** y comprueba que la base de datos lo rechaza.

Código y resultado: el problema

```
CREATE TABLE hoja_pedidos (  
  cliente text,  
  telefono text,  
  pedido integer,  
  total numeric  
);  
  
INSERT INTO hoja_pedidos VALUES  
  ('Ana García', '600111222', 101, 45.90),  
  ('Ana García', '600111222', 103, 80.50),  
  ('Ana Garcia', '600111223', 110, 20.00);  
  
SELECT DISTINCT cliente, telefono  
FROM hoja_pedidos  
ORDER BY cliente;
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE hoja_pedidos (
ensayo-#   cliente text,
ensayo-#   telefono text,
ensayo-#   pedido integer,
ensayo-#   total numeric
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO hoja_pedidos VALUES
ensayo-#   ('Ana García', '600111222', 101, 45.90),
ensayo-#   ('Ana García', '600111222', 103, 80.50),
ensayo-#   ('Ana Garcia', '600111223', 110, 20.00);
INSERT 0 3

ensayo=# SELECT DISTINCT cliente, telefono
ensayo-# FROM hoja_pedidos
ensayo-# ORDER BY cliente;
 cliente | telefono
-----+-----
 Ana Garcia | 600111223
 Ana García | 600111222
(2 rows)

```

Figura 1.2-a. Captura real: una tabla sin reglas, como una hoja de cálculo, guarda datos contradictorios sin avisar.

Qué ves en la imagen.

1. `CREATE TABLE hoja_pedidos (...)` crea una tabla con cuatro columnas de texto o número y ninguna regla. Hace lo mismo que una hoja de cálculo: acepta lo que le des.
2. `INSERT ... VALUES` guarda tres filas con un mismo cliente. En la tercera hay dos pequeños descuidos: «Garcia» sin tilde y un teléfono con la última cifra distinta.
3. `SELECT DISTINCT cliente, telefono` pide las combinaciones **distintas** de cliente y teléfono (`DISTINCT` quita las repetidas). Si todo estuviera bien, saldría una sola fila. Salen dos: para el sistema, hay **dos clientes** distintos. ¿Cuál es el teléfono correcto de Ana? No hay manera de saberlo.

pedidos.xlsx — Hoja de cálculo

	A · Cliente	B · Teléfono	C · Pedido	D · Total
1	Ana García	600111222	101	45,90
2	Ana García	600111222	103	80,50
3	Ana Garcia	600111223	110	20,00

☐ Teléfono distinto: ¿cuál es el correcto? ☐ Nombre escrito de otra forma

Figura 1.2-b. Ilustración: el mismo problema visto en una hoja de cálculo.

Qué ves en la imagen. Es la misma situación en una hoja de cálculo: tres filas para Ana, un teléfono distinto resaltado en rojo y un nombre escrito de otra forma resaltado en amarillo.

Código y resultado: la solución

```
CREATE TABLE clientes_demo (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  telefono text
);

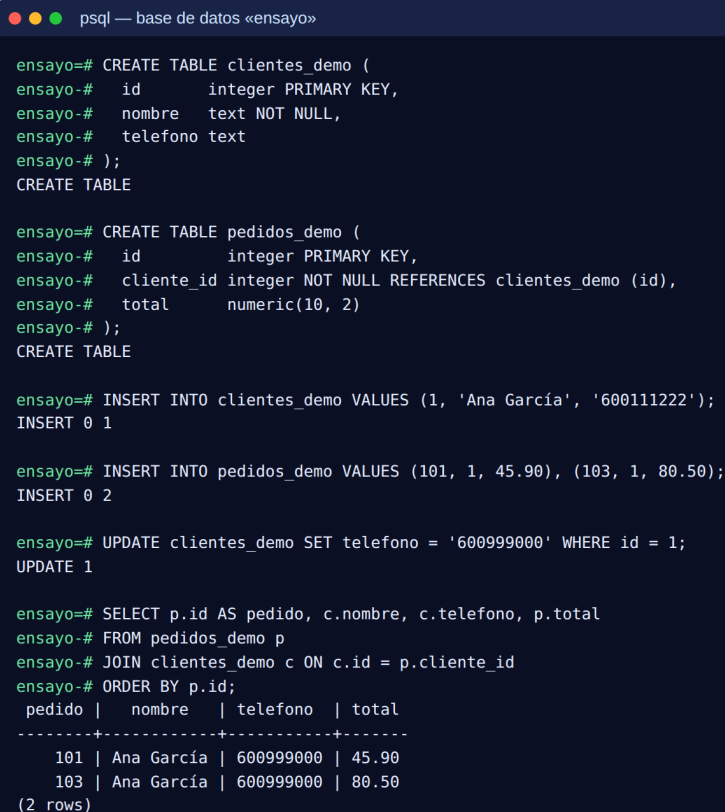
CREATE TABLE pedidos_demo (
  id          integer PRIMARY KEY,
  cliente_id integer NOT NULL REFERENCES clientes_demo (id),
  total       numeric(10, 2)
);

INSERT INTO clientes_demo VALUES (1, 'Ana García', '600111222');

INSERT INTO pedidos_demo VALUES (101, 1, 45.90), (103, 1, 80.50);

UPDATE clientes_demo SET telefono = '600999000' WHERE id = 1;

SELECT p.id AS pedido, c.nombre, c.telefono, p.total
FROM pedidos_demo p
JOIN clientes_demo c ON c.id = p.cliente_id
ORDER BY p.id;
```



```
psql — base de datos «ensayo»

ensayo=# CREATE TABLE clientes_demo (
ensayo=#   id      integer PRIMARY KEY,
ensayo=#   nombre  text NOT NULL,
ensayo=#   telefono text
ensayo=# );
CREATE TABLE

ensayo=# CREATE TABLE pedidos_demo (
ensayo=#   id          integer PRIMARY KEY,
ensayo=#   cliente_id integer NOT NULL REFERENCES clientes_demo (id),
ensayo=#   total       numeric(10, 2)
ensayo=# );
CREATE TABLE

ensayo=# INSERT INTO clientes_demo VALUES (1, 'Ana García', '600111222');
INSERT 0 1

ensayo=# INSERT INTO pedidos_demo VALUES (101, 1, 45.90), (103, 1, 80.50);
INSERT 0 2

ensayo=# UPDATE clientes_demo SET telefono = '600999000' WHERE id = 1;
UPDATE 1

ensayo=# SELECT p.id AS pedido, c.nombre, c.telefono, p.total
ensayo=# FROM pedidos_demo p
ensayo=# JOIN clientes_demo c ON c.id = p.cliente_id
ensayo=# ORDER BY p.id;
 pedido | nombre  | telefono | total
-----+-----+-----+-----
    101 | Ana García | 600999000 | 45.90
    103 | Ana García | 600999000 | 80.50
(2 rows)
```

Figura 1.2-c. Captura real: con dos tablas enlazadas, el teléfono de Ana se guarda y se cambia en un único sitio.

Qué ves en la imagen.

1. Se crean `clientes_demo` (cada cliente una sola vez) y `pedidos_demo`, cuya columna `cliente_id` apunta al cliente (`REFERENCES clientes_demo (id)`). Los detalles de `PRIMARY KEY` y `REFERENCES` se explican en el punto 1.7.
2. Se insertan un cliente y dos pedidos suyos. Los pedidos no repiten el nombre ni el teléfono: solo guardan el número 1.
3. `UPDATE clientes_demo SET telefono = '600999000' WHERE id = 1` cambia el teléfono. La respuesta `UPDATE 1` confirma que se modificó **una** fila.
4. La última consulta une las dos tablas con `JOIN` (se estudia en la Parte 5) para mostrar cada pedido junto a los datos del cliente. Los dos pedidos muestran el teléfono nuevo, porque lo leen del único sitio donde está guardado.

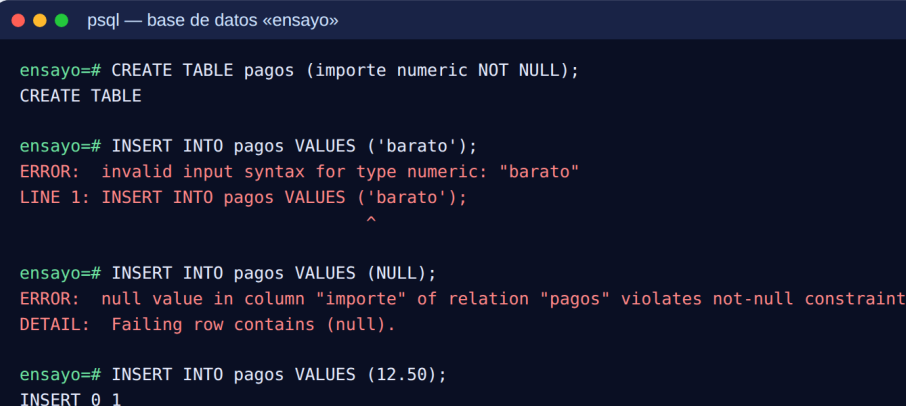
Código y resultado: la base de datos rechaza lo absurdo

```
CREATE TABLE pagos (importe numeric NOT NULL);

INSERT INTO pagos VALUES ('barato');

INSERT INTO pagos VALUES (NULL);

INSERT INTO pagos VALUES (12.50);
```



```
psql — base de datos «ensayo»

ensayo=# CREATE TABLE pagos (importe numeric NOT NULL);
CREATE TABLE

ensayo=# INSERT INTO pagos VALUES ('barato');
ERROR:  invalid input syntax for type numeric: "barato"
LINE 1: INSERT INTO pagos VALUES ('barato');
                                   ^

ensayo=# INSERT INTO pagos VALUES (NULL);
ERROR:  null value in column "importe" of relation "pagos" violates not-null constraint
DETAIL:  Failing row contains (null).

ensayo=# INSERT INTO pagos VALUES (12.50);
INSERT 0 1
```

Figura 1.2-d. Captura real: tipos y reglas que rechazan datos incorrectos.

Qué ves en la imagen.

1. `CREATE TABLE pagos (importe numeric NOT NULL)` crea una columna de importes que solo admite números y que no puede quedarse vacía.
2. Intentar guardar el texto `'barato'` produce `ERROR: invalid input syntax for type numeric: "barato"` («texto no válido para el tipo numérico»). `psql` incluso señala con una flecha `^` dónde está el problema.
3. Intentar guardar un valor vacío (`NULL`) produce `violates not-null constraint` («incumple la restricción de no vacío»).

4. Un importe correcto, `12.50`, se guarda (`INSERT 0 1`).

Ejemplo sencillo

En una hoja de cálculo de pedidos, el cliente «Ana García» aparece en tres filas y en una el teléfono está mal. ¿Cuál es el correcto? La hoja no puede decírtelo.

Ejemplo real

Una tienda con 20.000 pedidos de 3.000 clientes guarda los datos de Ana una sola vez en la tabla de clientes, y sus pedidos apuntan a ella. Si cambia de teléfono, se corrige en un único sitio y todos sus pedidos quedan al día.

Profundizando

Uso simultáneo. Cuando dos personas modifican el mismo dato a la vez, el gestor las pone en orden: gestiona los accesos simultáneos mediante un mecanismo de **transacciones** y bloqueos, de modo que una modificación no pisa a la otra. Lo veremos en la Parte 5.

Cuándo Sí es mejor una hoja de cálculo. Para un análisis puntual, un presupuesto personal o datos que usa una sola persona, una hoja de cálculo es más rápida y cómoda. La base de datos compensa cuando hay varios usuarios, relaciones entre datos, reglas que cumplir o aplicaciones que necesitan acceder.

Persistencia. «Guardar de forma fiable» significa que, cuando el gestor responde «hecho», el dato sobrevive aunque el ordenador se apague justo después. Los gestores lo consiguen escribiendo un registro de cambios en disco antes de confirmar.

Ejercicio

1. Con la tabla `hoja_pedidos` de la figura 1.2-a, escribe una consulta que cuente cuántos teléfonos distintos hay (usa `count(DISTINCT telefono)`).
2. Explica en una frase qué ocurriría si la hoja de cálculo la usan cinco personas a la vez.
3. Piensa en una lista que gestionas con una hoja de cálculo. Escribe una cosa que te gustaría poder impedir y di con qué la impedirías en una base de datos.

Solución

1. `SELECT count(DISTINCT telefono) FROM hoja_pedidos;` devuelve 2: los dos teléfonos distintos (`600111222` y `600111223`) para lo que debería ser un solo cliente.
2. Una persona podría guardar sus cambios encima de los de otra sin que nadie lo note. El gestor coordina los accesos para que no ocurra.
3. Por ejemplo, impedir que se escriba texto en una columna de importes: se impide con el tipo de dato (`numeric`) y con restricciones como `NOT NULL` o `CHECK` (puntos 1.5 y 3.5).

Error habitual

Pensar que una base de datos es «una hoja de cálculo más grande». Su valor no está en el tamaño, sino en las reglas y en las relaciones entre tablas.

```
-- Una tabla sin reglas: no es mejor que una hoja de cálculo  
CREATE TABLE todo (cosa1 text, cosa2 text, cosa3 text);
```

Buena práctica

Guarda cada dato una sola vez y enlázalo desde donde se necesite. Cuando veas el mismo dato escrito en varias filas, pregúntate si debería vivir en una tabla propia.

Comprobación

Sin mirar el texto, nombra tres problemas de una hoja de cálculo que una base de datos resuelve y explica cómo. Después, reproduce en `psql` el ejemplo de `hoja_pedidos` y verifica que `DISTINCT` te muestra la contradicción.

1.3 Qué es una base de datos y qué es un gestor

¿Qué es?

Una **base de datos** es un conjunto organizado de datos relacionados entre sí, guardados de manera que se puedan consultar, modificar y proteger.

Un **sistema gestor de bases de datos** (abreviado **SGBD**) es el programa que crea, guarda, protege y consulta esos datos. Ejemplos de gestores son PostgreSQL, MySQL, SQLite y SQL Server.

Son dos cosas distintas, y mucha gente las confunde:

Concepto	Qué es	Ejemplo
Gestor (SGBD)	Un programa	PostgreSQL
Base de datos	Un conjunto de datos organizados	La base tienda con clientes y pedidos

¿Para qué sirve el gestor?

Es el **intermediario** entre las aplicaciones y los datos guardados en disco. Las aplicaciones no abren los archivos de datos directamente: le piden las cosas al gestor y este las hace, aplicando las reglas y comprobando los permisos.

¿Por qué lo necesito?

Te permite distinguir dos frases que parecen iguales: «he instalado PostgreSQL» (el gestor) y «he creado la base de datos **tienda**» (los datos). Un mismo gestor puede contener muchas bases de datos independientes. Si no distingues ambas cosas, te resultará confuso leer documentación, pedir ayuda o configurar una conexión.

¿Cómo funciona?

Cuando una aplicación necesita un dato, ocurre este recorrido:

1. La aplicación envía una **petición** al gestor (una consulta).
2. El gestor comprueba que quien pregunta tiene **permiso**.
3. El gestor busca o modifica los datos en los **archivos** del disco.
4. El gestor devuelve el **resultado** a la aplicación.

La aplicación nunca toca los archivos. Eso garantiza que las reglas se cumplan siempre, venga la petición de la web, de la app o de una automatización.

Paso a paso: ver el gestor y sus bases de datos

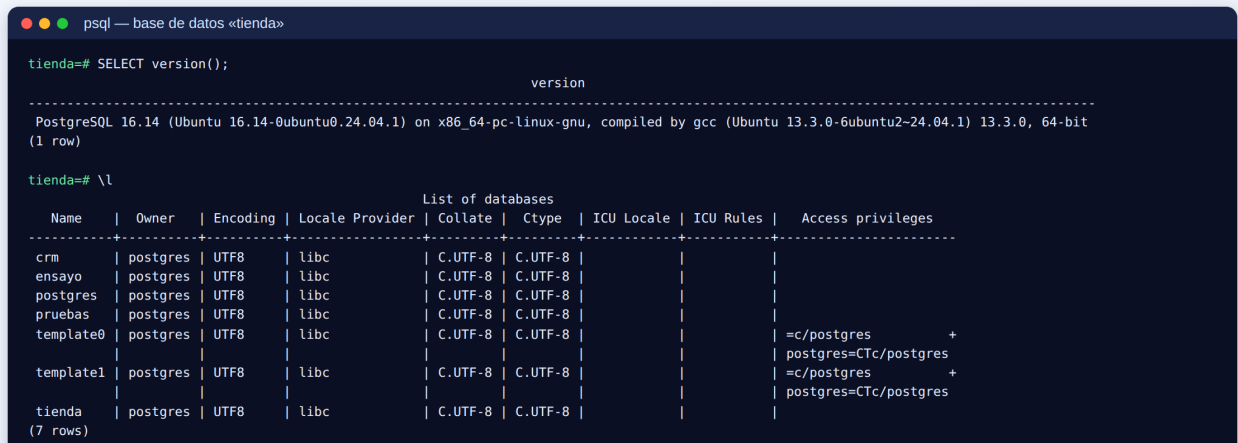
1. **Pregunta qué gestor tienes:** `SELECT version();` te dice el programa y su versión.
2. **Lista las bases de datos** que contiene con `\l`.

3. **Comprueba que son independientes:** crea una tabla en una base de datos y mira otra distinta.
4. **Comprueba que el gestor aplica permisos:** crea un usuario sin permisos, intenta leer una tabla y observa el rechazo; concede el permiso y repite.

Código y resultado: el gestor y sus bases de datos

```
SELECT version();
```

```
\l
```



The screenshot shows a terminal window titled 'psql — base de datos «tienda»'. The first command executed is 'tienda=# SELECT version();', which returns the PostgreSQL version: 'PostgreSQL 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0, 64-bit (1 row)'. The second command is 'tienda=# \l', which lists the databases. The output is a table with columns: Name, Owner, Encoding, Locale Provider, Collate, Ctype, ICU Locale, ICU Rules, and Access privileges. The listed databases are: crm, ensayo, postgres, pruebas, template0, template1, and tienda. The 'postgres' database has special access privileges for the user 'postgres'.

Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges
crm	postgres	UTF8	libc	C.UTF-8	C.UTF-8			
ensayo	postgres	UTF8	libc	C.UTF-8	C.UTF-8			
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8			
pruebas	postgres	UTF8	libc	C.UTF-8	C.UTF-8			
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			postgres=CTc/postgres +
tienda	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +
								postgres=CTc/postgres

Figura 1.3-a. Captura real: la versión del gestor y la lista de bases de datos que contiene.

Qué ves en la imagen.

1. `SELECT version()` devuelve el texto «PostgreSQL 16.14 ...»: el programa es PostgreSQL, en su versión 16.14.
2. `\l` (de *list*, «listar») muestra **una sola** instalación de PostgreSQL con **siete bases de datos**: `crm`, `ensayo`, `postgres`, `pruebas`, `tienda`, y dos más, `template0` y `template1`. Estas dos últimas son **plantillas** que PostgreSQL usa internamente para crear bases de datos nuevas: no las toques. `postgres` es la base de datos que existe desde el principio. Las columnas son: **Name** (nombre), **Owner** (propietario), **Encoding** (cómo se guardan los caracteres: `UTF8` admite tildes y símbolos de cualquier idioma) y el resto son detalles de ordenación e idioma que veremos en la Parte 6.

El gestor es uno; las bases de datos son muchas.

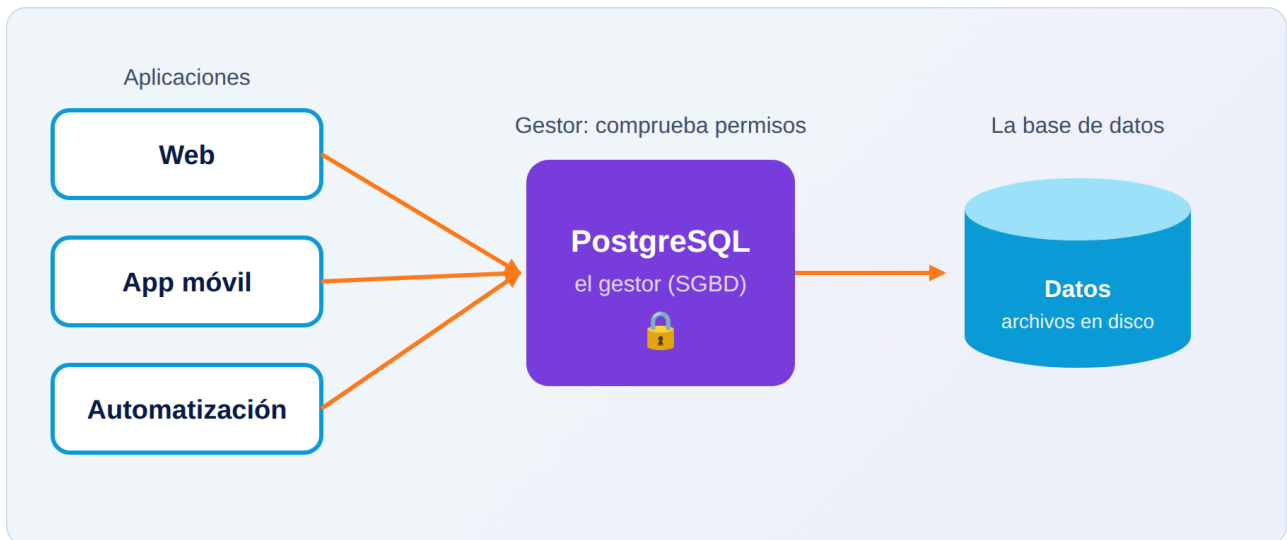


Figura 1.3-b. Ilustración: el gestor está entre las aplicaciones y los datos.

Qué ves en la imagen. A la izquierda, tres aplicaciones que piden datos: una web, una app móvil y una automatización. En el centro, el gestor, con un candado que representa los permisos. A la derecha, los archivos en disco. Todas las flechas pasan por el gestor.

Código y resultado: las bases de datos son independientes

```
CREATE TABLE empresas (id integer PRIMARY KEY, nombre text);

INSERT INTO empresas VALUES (1, 'Acme S.L.');
```

`\dt`

```
psql — base de datos «crm»

crm=# CREATE TABLE empresas (id integer PRIMARY KEY, nombre text);
CREATE TABLE

crm=# INSERT INTO empresas VALUES (1, 'Acme S.L.');
```

```
INSERT 0 1

crm=# \dt
```

Schema	Name	Type	Owner
public	empresas	table	postgres

(1 row)

Figura 1.3-c. Captura real: la base de datos `crm` contiene una tabla `empresas`.

Qué ves en la imagen. Conectados a `crm`, se crea la tabla `empresas`, se inserta una fila y `\dt` (de *describe tables*, «listar tablas») confirma que `crm` contiene esa tabla.

`\dt`

```
psql — base de datos «pruebas»

pruebas=# \dt
Did not find any relations.
```

Figura 1.3-d. Captura real: la base de datos `pruebas` no contiene ninguna tabla.

Qué ves en la imagen. Ahora estamos conectados a `pruebas`. `\dt` responde `Did not find any relations.` («no se ha encontrado ninguna relación»). La tabla `empresas` está en `crm`, no aquí: son bases de datos independientes aunque las gestione el mismo programa.

Gestor PostgreSQL (un solo programa)

Base de datos: tienda

tabla clientes
tabla pedidos
tabla productos

Base de datos: crm

tabla empresas
tabla contactos
tabla tareas

Base de datos: pruebas

tabla borrador

Figura 1.3-e. Ilustración: un gestor con tres bases de datos independientes.

Qué ves en la imagen. Un único gestor (el recuadro morado) contiene `tienda`, `crm` y `pruebas`, cada una con sus propias tablas.

Código y resultado: el gestor aplica permisos

```
psql -h /tmp -p 5433 -U postgres -d crm -c "CREATE ROLE lector LOGIN;"
psql -h /tmp -p 5433 -U lector -d crm -c "SELECT * FROM empresas;"
psql -h /tmp -p 5433 -U postgres -d crm -c "GRANT SELECT ON empresas TO lector;"
psql -h /tmp -p 5433 -U lector -d crm -c "SELECT * FROM empresas;"
```

```
Terminal

$ psql -h /tmp -p 5433 -U postgres -d crm -c "CREATE ROLE lector LOGIN;"
CREATE ROLE
$ psql -h /tmp -p 5433 -U lector -d crm -c "SELECT * FROM empresas;"
ERROR:  permission denied for table empresas
$ psql -h /tmp -p 5433 -U postgres -d crm -c "GRANT SELECT ON empresas TO lector;"
GRANT
$ psql -h /tmp -p 5433 -U lector -d crm -c "SELECT * FROM empresas;"
 id | nombre
-----+-----
  1 | Acme S.L.
(1 row)
```

Figura 1.3-f. Captura real: un usuario sin permisos no puede leer una tabla; con el permiso, sí.

Qué ves en la imagen. Cuatro órdenes ejecutadas desde la terminal (las opciones `-h`, `-p`, `-U` y `-d` indican dónde está el servidor, el puerto, el usuario y la base de datos; en tu instalación pueden ser otras):

1. `CREATE ROLE lector LOGIN;` crea un usuario llamado `lector` que puede conectarse (`LOGIN`), pero que no tiene permiso sobre nada.
2. Como `lector`, `SELECT * FROM empresas;` falla con `ERROR: permission denied for table empresas` («permiso denegado»). El gestor ha comprobado quién pregunta y ha dicho que no.
3. Como propietario, `GRANT SELECT ON empresas TO lector;` concede el permiso de lectura (`GRANT` significa «conceder»).
4. De nuevo como `lector`, la consulta funciona y devuelve la fila de Acme S.L.

Esto es exactamente el paso 2 del recorrido: **el gestor comprueba los permisos antes de tocar los datos**. Los usuarios y permisos se explican en profundidad en la Parte 6.

Ejemplo sencillo

Piensa en una biblioteca. Los libros y las fichas son la **base de datos**. La bibliotecaria que busca, presta y recoge es el **gestor**. Tú eres la **aplicación**: no entras al almacén, se lo pides a ella.

Ejemplo real

Un servidor tiene PostgreSQL instalado y dentro hay tres bases de datos: `tienda`, `crm` y `pruebas`. El gestor es uno; las bases de datos son tres. Una aplicación puede tener acceso a `tienda` y ninguno a `crm`.

Profundizando

Servidor, base de datos, esquema, tabla. La jerarquía completa es: un **servidor** (el programa) contiene **bases de datos**; cada base de datos contiene **esquemas** (carpetas de tablas; `public` es el esquema por defecto); cada esquema contiene **tablas**. En `\dt` verás la columna `Schema` con el valor `public`. Lo veremos en la Parte 6.

Otros gestores. MySQL, MariaDB, SQLite y SQL Server hacen el mismo papel con diferencias de detalle. En la Parte 13 los comparamos.

Gestor y archivos. PostgreSQL guarda sus datos en una carpeta del disco con archivos propios. Nunca debes editarlos ni copiarlos a mano mientras el gestor está en marcha: para eso existen las copias de seguridad (Parte 19).

Ejercicio

1. Clasifica cada elemento como «gestor» o «base de datos»: PostgreSQL, la base `crm`, MySQL, la base `tienda`.
2. Completa la frase: «Para guardar mis clientes primero instalo ____ y después creo ____».
3. ¿Qué comando de `psql` te dice qué bases de datos existen? ¿Y qué tablas tiene la base a la que estás conectado?

Solución

1. Gestor: PostgreSQL y MySQL. Base de datos: `crm` y `tienda`.
2. «Para guardar mis clientes primero instalo **un gestor (por ejemplo, PostgreSQL)** y después creo **una base de datos**».
3. `\l` lista las bases de datos y `\dt` las tablas de la base actual.

Error habitual

Decir «base de datos PostgreSQL» para todo. PostgreSQL es el gestor; tus datos viven en bases de datos dentro de él. Y creer que una tabla creada en una base de datos «se ve» desde otra: no es así.

```
-- Conectado a la base `pruebas`, esta tabla no existe aunque exista en `crm`  
SELECT * FROM empresas; -- ERROR: relation "empresas" does not exist
```

Buena práctica

Usa cada palabra con su significado. Cuando hables o escribas, pregúntate: ¿me refiero al programa o a los datos?

Comprobación

Explica con tus palabras la diferencia entre gestor y base de datos. Después, con `\l` en `psql`, comprueba cuántas bases de datos tiene tu instalación y cuáles son plantillas.

1.4 Anatomía de una tabla

¿Qué es?

Una **tabla** es la forma de organizar los datos de un mismo tipo de cosa: clientes, pedidos, productos. Se parece a una cuadrícula de filas y columnas, como una hoja de cálculo, pero con reglas más estrictas.

Estas son sus partes:

- **Tabla:** el conjunto completo. Tiene un nombre, por ejemplo `clientes`.
- **Columna:** una característica que tienen todos los elementos. En `clientes` son `id`, `nombre`, `email` y `telefono`. Una columna también se llama **campo**: «campo» y «columna» son dos nombres para lo mismo.
- **Fila:** un elemento concreto de la tabla, por ejemplo un cliente. Una fila también se llama **registro**. Los dos nombres significan lo mismo.
- **Valor:** el dato que hay en el cruce de una fila y una columna, por ejemplo `luis@ejemplo.com`.
- **Cabecera:** la primera fila, que contiene los nombres de las columnas. No es un registro, solo rotula las columnas.

¿Para qué sirve?

Da una estructura fija. Todos los clientes tienen los mismos campos, así que se pueden buscar, ordenar, contar y comparar sin esfuerzo. Si cada cliente se guardase con campos distintos, no podrías preguntar «dame todos los emails».

¿Por qué lo necesito?

Todo lo que harás con una base de datos (diseñar, consultar, modificar, proteger) se apoya en tablas. Si no tienes claro qué es una fila y qué es una columna, el resto resultará confuso.

¿Cómo funciona?

Una tabla se define una vez: se decide su nombre, sus columnas y el tipo de dato de cada una. Después se van añadiendo filas. Cuando llega un cliente nuevo, se añade una fila; cuando se da de baja, se elimina. Las columnas no cambian con cada cliente: son la plantilla.

Una regla básica: **cada fila describe una sola cosa, y cada columna guarda una sola característica**. En una fila de `clientes` va un cliente, no dos; y el campo `email` guarda un email, no un email y un teléfono mezclados.

Primero, sin código: construye la tabla a mano

Antes de escribir una sola línea de código, una tabla se construye **con papel y lápiz** (o en una hoja de cálculo). Es la mejor forma de entenderla.

1. **Decide qué cosa guardas** (clientes) y **dibuja una cuadrícula**. Escribe en la primera fila el nombre de cada característica: esas son las **columnas**.
2. **Añade una fila por cada cliente**. Escribe cada dato bajo su columna. Si no sabes un dato, deja la celda vacía (en una base de datos eso será NULL).
3. **Comprueba**: ¿cada fila es un cliente? ¿cada columna guarda un dato? ¿Puedes señalar con el dedo una fila, una columna y un valor?

1
Decide qué cosa guardas y dibuja las columnas

clientes			
id	nombre	email	telefono

Una columna por cada característica. La primera fila (cabecera) solo pone los nombres.

2
Añade una fila por cada cliente

clientes			
id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

En amarillo, lo que se acaba de escribir. Una celda sin dato se deja vacía: eso es NULL.

3
Comprueba: ¿una fila = una cosa? ¿una columna = un dato?

clientes			
id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

Fila de Luis (naranja) = un cliente. Columna email (azul) = un dato por celda. Cruce = el valor luis@ejemplo.com.

Figura 1.4-a. Sin código: construir la tabla CLIENTES a mano, en tres pasos.

Qué ves en la imagen. Paso 1: solo las columnas. Paso 2: las filas añadidas (en amarillo, lo que se acaba de escribir; la celda vacía de Luis es su NULL). Paso 3: la lectura: la fila de Luis es un registro, la columna `email` es un campo y su cruce es un valor. **Si entiendes esta figura, ya sabes lo que es una tabla.** Lo que sigue es hacer lo mismo, pero dentro de PostgreSQL.

Paso a paso: ahora con código — crear y leer la tabla `clientes``

1. **Conéctate a la base de datos** `tienda` con `\c tienda` (si no existe, créala antes con `CREATE DATABASE tienda;`, punto 0.5).
2. **Decide las columnas**: `id`, `nombre`, `email`, `telefono` (las mismas que dibujaste).
3. **Crea la tabla** con `CREATE TABLE`. Verás la respuesta `CREATE TABLE`.
4. **Inserta tres filas** con `INSERT`. Verás `INSERT 0 3`.
5. **Léela** con `SELECT * FROM clientes;` y comprueba que ves tres filas y cuatro columnas.

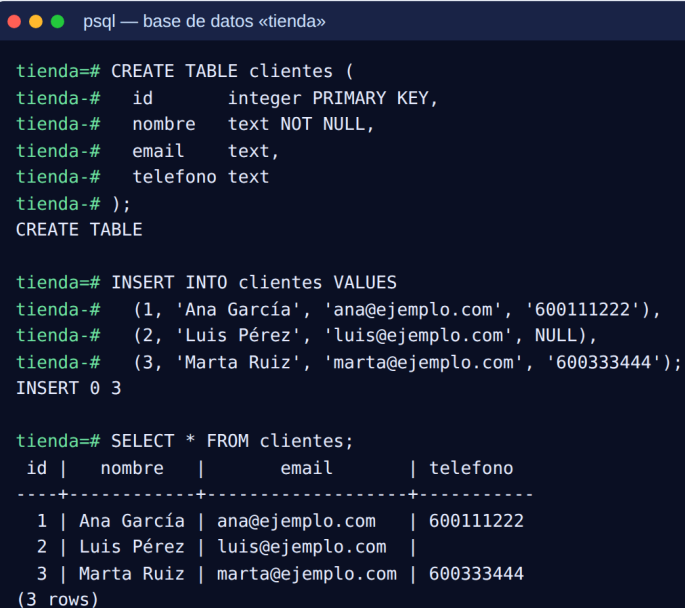
6. Describe su estructura con `\d clientes` y compara con lo que decidiste.
7. Localiza un valor concreto y cuenta filas y columnas con consultas.

Código y resultado

```
CREATE TABLE clientes (
  id      integer PRIMARY KEY,
  nombre  text NOT NULL,
  email   text,
  telefono text
);

INSERT INTO clientes VALUES
(1, 'Ana García', 'ana@ejemplo.com', '600111222'),
(2, 'Luis Pérez', 'luis@ejemplo.com', NULL),
(3, 'Marta Ruiz', 'marta@ejemplo.com', '600333444');

SELECT * FROM clientes;
```



```
psql — base de datos «tienda»

tienda=# CREATE TABLE clientes (
tienda-#   id      integer PRIMARY KEY,
tienda-#   nombre  text NOT NULL,
tienda-#   email   text,
tienda-#   telefono text
tienda-# );
CREATE TABLE

tienda=# INSERT INTO clientes VALUES
tienda-# (1, 'Ana García', 'ana@ejemplo.com', '600111222'),
tienda-# (2, 'Luis Pérez', 'luis@ejemplo.com', NULL),
tienda-# (3, 'Marta Ruiz', 'marta@ejemplo.com', '600333444');
INSERT 0 3

tienda=# SELECT * FROM clientes;
 id | nombre  | email           | telefono
-----+-----+-----+-----
  1 | Ana García | ana@ejemplo.com | 600111222
  2 | Luis Pérez | luis@ejemplo.com | 
  3 | Marta Ruiz | marta@ejemplo.com | 600333444
(3 rows)
```

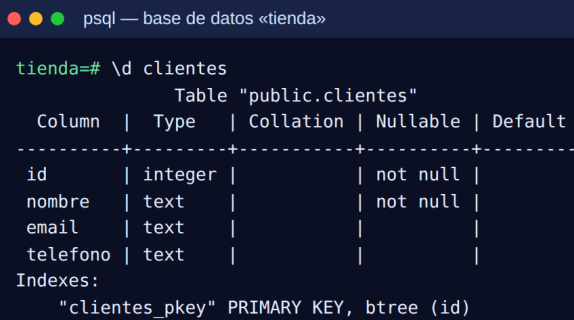
Figura 1.4-b. Captura real: creación de la tabla `clientes`, inserción de tres filas y lectura.

Qué ves en la imagen, orden por orden.

1. `CREATE TABLE clientes (...)`: cada línea entre paréntesis define una columna. `id integer PRIMARY KEY` es un número entero que identifica la fila (punto 1.7); `nombre text NOT NULL` es texto obligatorio; `email text` y `telefono text` son textos opcionales.
2. `INSERT INTO clientes VALUES (...), (...), (...)`: inserta tres filas de una vez, separadas por comas. Los valores van en el mismo orden que las columnas. A Luis le ponemos `NULL` en el teléfono: no sabemos su número (punto 1.6).

3. `SELECT * FROM clientes` devuelve **tres filas** y **cuatro columnas**. La cabecera (`id | nombre | email | telefono`) son los nombres de las columnas. Cada línea siguiente es un registro. La celda vacía de Luis es su NULL.

```
\d clientes
```



```
tienda=# \d clientes
          Table "public.clientes"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
  id      | integer |           | not null |
  nombre  | text    |           | not null |
  email   | text    |           |          |
  telefono | text    |           |          |
Indexes:
  "clientes_pkey" PRIMARY KEY, btree (id)
```

Figura 1.4-c. Captura real: la estructura de la tabla, con `\d`.

Qué ves en la imagen. `\d clientes` (la «d» viene de *describe*, «describir») muestra cada columna con su **Type** (tipo), si es obligatoria (**Nullable**: `not null`) y los índices (**Indexes**): `clientes_pkey` es la clave primaria, que PostgreSQL ha creado automáticamente.

```
SELECT email FROM clientes WHERE nombre = 'Marta Ruiz';

SELECT count(*) AS filas FROM clientes;

SELECT count(*) AS columnas
FROM information_schema.columns
WHERE table_name = 'clientes';
```

```

psql — base de datos «tienda»

tienda=# SELECT email FROM clientes WHERE nombre = 'Marta Ruiz';
      email
-----
 marta@ejemplo.com
(1 row)

tienda=# SELECT count(*) AS filas FROM clientes;
      filas
-----
         3
(1 row)

tienda=# SELECT count(*) AS columnas
tienda=# FROM information_schema.columns
tienda=# WHERE table_name = 'clientes';
      columnas
-----
         4
(1 row)

```

Figura 1.4-d. Captura real: localizar un valor y contar filas y columnas.

Qué ves en la imagen.

1. `SELECT email FROM clientes WHERE nombre = 'Marta Ruiz'` busca la **fila** de Marta (`WHERE` significa «donde») y devuelve el **valor** de su columna `email`: `marta@ejemplo.com`. Ese es el cruce de fila y columna.
2. `SELECT count(*) AS filas FROM clientes` cuenta las filas: 3. (`AS filas` pone nombre a la columna del resultado.)
3. La tercera consulta cuenta las columnas leyendo el catálogo, la parte del sistema que guarda la descripción de las tablas: 4.

tienda · clientes			
id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

☐ Fila = registro (Luis)
 ☐ Columna = campo (email)
 ☐ Valor

Figura 1.4-e. Ilustración: la fila naranja es un registro; la columna azul es un campo.

Qué ves en la imagen. La fila de Luis, en naranja, es un **registro** completo. La columna `email`, en azul, es un **campo**. La celda en la que se cruzan, `luis@ejemplo.com`, es un **valor**.

Elemento	Qué representa en CLIENTES
Tabla	Todos los clientes del negocio
Columna (campo)	Una característica: <code>id</code> , <code>nombre</code> , <code>email</code> , <code>telefono</code>
Fila (registro)	Un cliente concreto, por ejemplo Luis
Valor	Un dato concreto, por ejemplo <code>luis@ejemplo.com</code>

Ejemplo sencillo

La tabla CLIENTES con cuatro columnas y tres filas. Añadir un cliente es añadir una fila; añadir una característica nueva (por ejemplo, la ciudad) sería añadir una columna a toda la tabla.

Ejemplo real

Una tienda tiene la tabla `clientes` con 3.000 filas. Al abrir tu cuenta, la aplicación busca tu fila, lee tus valores y los muestra en pantalla. Cuando cambias tu teléfono, modifica un único valor en tu fila.

Profundizando

El orden de las filas no está garantizado. Una tabla es un conjunto de filas: sin pedir un orden explícito (`ORDER BY` , punto 1.9), el gestor las devuelve en el orden que le resulte más cómodo, y puede cambiar. No te fíes del orden en el que las insertaste.

El orden de las columnas sí importa al insertar. `INSERT INTO clientes VALUES (...)` usa el orden de las columnas. Una forma más segura es indicar los nombres: `INSERT INTO clientes (id, nombre) VALUES (4, 'Pedro')` ; las columnas no mencionadas quedan en NULL.

Cuántas columnas y filas. Una tabla puede tener millones de filas. El número de columnas suele ser pequeño (de unas pocas a unas decenas). Si una tabla tiene cientos de columnas, probablemente esté mal diseñada (Parte 4).

Ejercicio

1. Escribe el `CREATE TABLE` de una tabla `productos` con las columnas `id` , `nombre` y `precio_eur` , y el `INSERT` de dos productos.
2. En la figura 1.4-b, ¿cuántas filas y cuántas columnas hay? ¿Qué valor está en la fila de Marta, columna `email` ?
3. ¿Qué está mal en esta fila de `clientes` : `4 | Pedro Gil y Laura Sanz | pedro@ejemplo.com / laura@ejemplo.com | 600555666` ?

Solución

1. Una posible solución:

```
CREATE TABLE productos (
  id          integer PRIMARY KEY,
  nombre      text NOT NULL,
  precio_eur  numeric(10, 2)
);
INSERT INTO productos VALUES (1, 'Camiseta', 19.95), (2, 'Gorra', 9.90);
```

1. Hay 3 filas (sin contar la cabecera) y 4 columnas. El valor es `marta@ejemplo.com`.
2. La fila mezcla dos personas (Pedro y Laura) y dos emails en una sola celda. Debería haber una fila por persona y un email por celda (se estudia en la Parte 4).

Error habitual

Confundir **tabla** con **base de datos**. Una base de datos contiene muchas tablas; una tabla guarda un solo tipo de cosa. Y poner en una misma celda varios datos separados por comas.

```
-- Mal: dos emails en una celda; no se puede buscar por cada uno
INSERT INTO clientes VALUES (4, 'Pedro', 'pedro@ejemplo.com / otro@ejemplo.com',
NULL);
```

Buena práctica

Nombra las tablas en plural y en minúsculas (`clientes`) y las columnas en singular, en minúsculas y sin espacios ni tildes (`nombre`, `fecha_alta`). Usa siempre el mismo estilo.

Comprobación

Con `\d tabla` comprueba que la estructura es la que pensaste. Con `SELECT count()` comprueba que el número de filas es el esperado. Si dudas, vuelve a leer la tabla con `SELECT`.

1.5 Tipos de datos

¿Qué es?

El **tipo de dato** indica qué clase de valor admite una columna. Los tipos básicos son:

- **Texto**: letras, números y símbolos tratados como texto (nombres, emails, descripciones).
- **Entero**: números sin decimales (cantidades, edades, contadores).
- **Decimal**: números con decimales (precios, medidas).
- **Fecha o fecha y hora**: un momento en el calendario.
- **Verdadero/falso** (también llamado **booleano**): solo admite dos valores, sí o no.

¿Para qué sirve?

Evita errores y habilita cálculos. Una columna de precios que solo admite números impide que alguien escriba «barato». Y como el gestor sabe que son números, puede sumarlos, compararlos y ordenarlos correctamente.

¿Por qué lo necesito?

Si guardas los precios como texto, el gestor los ordena como si fueran palabras, y «100» queda antes que «20», porque compara carácter a carácter y «1» va antes que «2». Con el tipo correcto, 20 va antes que 100 y puedes calcular la suma de ventas del mes. El tipo no es un detalle: determina lo que podrás hacer con el dato.

¿Cómo funciona?

1. Al crear la tabla eliges un tipo para cada columna.
2. Cuando intentas guardar un valor, el gestor comprueba que encaja con el tipo.
3. Si no encaja, **rechaza** la operación y devuelve un error en lugar de guardar algo incorrecto.

Cada gestor da nombres propios a sus tipos. En PostgreSQL, los básicos son:

Tipo (nombre general)	Nombre en PostgreSQL	Ejemplo
Texto	<code>text</code>	<code>'Camiseta'</code>
Entero	<code>integer</code>	<code>12</code>
Decimal exacto	<code>numeric(10, 2)</code>	<code>19.95</code>
Fecha	<code>date</code>	<code>'2026-03-02'</code>
Fecha y hora con zona horaria	<code>timestampz</code>	<code>'2026-03-02 10:00:00+01'</code>
Verdadero/falso	<code>boolean</code>	<code>true</code>

Paso a paso: crear una tabla con todos los tipos y ponerlos a prueba

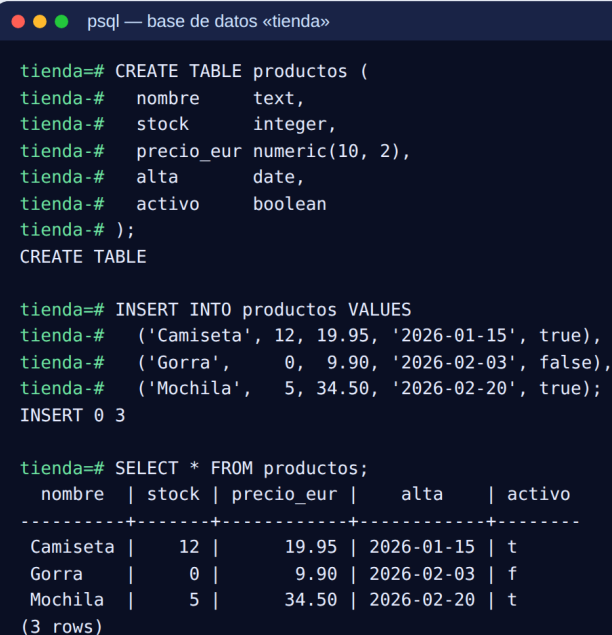
1. Crea la tabla `productos` con una columna de cada tipo.
2. Inserta tres productos válidos y léelos.
3. Intenta guardar un dato de tipo equivocado (texto en una columna entera) y lee el error.
4. Intenta guardar una fecha imposible (31 de febrero) y lee el error.
5. Pregunta a PostgreSQL qué tipo tiene cada columna con `pg_typeof`.
6. Comprueba por qué importa el tipo: ordena números guardados como texto y como número.

Código y resultado

```
CREATE TABLE productos (
  nombre      text,
  stock       integer,
  precio_eur  numeric(10, 2),
  alta        date,
  activo      boolean
);

INSERT INTO productos VALUES
  ('Camiseta', 12, 19.95, '2026-01-15', true),
  ('Gorra',    0,  9.90, '2026-02-03', false),
  ('Mochila',  5, 34.50, '2026-02-20', true);

SELECT * FROM productos;
```



The screenshot shows a PostgreSQL terminal window titled "psql — base de datos «tienda»". The user enters several SQL commands to create a table, insert data, and query it. The output shows the table structure and the inserted data in a tabular format.

```
tienda=# CREATE TABLE productos (
tienda-#  nombre      text,
tienda-#  stock       integer,
tienda-#  precio_eur  numeric(10, 2),
tienda-#  alta        date,
tienda-#  activo      boolean
tienda-# );
CREATE TABLE

tienda=# INSERT INTO productos VALUES
tienda-#  ('Camiseta', 12, 19.95, '2026-01-15', true),
tienda-#  ('Gorra',    0,  9.90, '2026-02-03', false),
tienda-#  ('Mochila',  5, 34.50, '2026-02-20', true);
INSERT 0 3

tienda=# SELECT * FROM productos;
 nombre | stock | precio_eur | alta      | activo
-----+-----+-----+-----+-----
Camiseta |   12 |    19.95 | 2026-01-15 | t
Gorra   |    0 |     9.90 | 2026-02-03 | f
Mochila  |    5 |    34.50 | 2026-02-20 | t
(3 rows)
```

Figura 1.5-a. Captura real: una tabla con un tipo de dato por columna.

Qué ves en la imagen.

1. `nombre text` : texto. `stock integer` : entero. `precio_eur numeric(10, 2)` : decimal exacto con hasta 10 cifras, 2 de ellas decimales. `alta date` : fecha. `activo boolean` : verdadero/falso.
2. Tras insertar tres productos, `SELECT *` los muestra. Fíjate en cómo se escribe cada tipo: las fechas entre comillas con el formato `año-mes-día`, los decimales con **punto**, y el booleano como `true` / `false` (que `psql` muestra como `t` y `f`).



nombre texto	stock entero	precio_eur decimal	alta fecha	activo verdadero/falso
Camiseta	12	19.95	2026-01-15	true
Gorra	0	9.90	2026-02-03	false
Mochila	5	34.50	2026-02-20	true

Figura 1.5-b. Ilustración: cada columna con su tipo.

Qué ves en la imagen. La misma tabla con una etiqueta de tipo sobre cada columna.

```
INSERT INTO productos (nombre, stock) VALUES ('Taza', 'muchos');

INSERT INTO productos (nombre, alta) VALUES ('Taza', '2026-02-31');

SELECT pg_typeof(nombre) AS nombre,
       pg_typeof(stock) AS stock,
       pg_typeof(precio_eur) AS precio_eur,
       pg_typeof(alta) AS alta,
       pg_typeof(activo) AS activo
FROM productos
LIMIT 1;
```

```

psql — base de datos «tienda»

tienda=# INSERT INTO productos (nombre, stock) VALUES ('Taza', 'muchos');
ERROR: invalid input syntax for type integer: "muchos"
LINE 1: ...RT INTO productos (nombre, stock) VALUES ('Taza', 'muchos');
                                     ^

tienda=# INSERT INTO productos (nombre, alta) VALUES ('Taza', '2026-02-31');
ERROR: date/time field value out of range: "2026-02-31"
LINE 1: ...ERT INTO productos (nombre, alta) VALUES ('Taza', '2026-02-31');
                                     ^

tienda=# SELECT pg_typeof(nombre) AS nombre,
tienda=#           pg_typeof(stock) AS stock,
tienda=#           pg_typeof(precio_eur) AS precio_eur,
tienda=#           pg_typeof(alta) AS alta,
tienda=#           pg_typeof(activo) AS activo
tienda=# FROM productos
tienda=# LIMIT 1;
 nombre | stock | precio_eur | alta | activo
-----+-----+-----+-----+-----
 text  | integer | numeric  | date | boolean
(1 row)

```

Figura 1.5-c. Captura real: PostgreSQL rechaza valores que no encajan con el tipo y confirma el tipo de cada columna.

Qué ves en la imagen.

1. Guardar el texto `'muchos'` en la columna `stock` (entera) da `ERROR: invalid input syntax for type integer: "muchos"`. Muestra incluso la línea de la orden y una flecha `^` donde está el problema.
2. Guardar `'2026-02-31'` en la columna de fecha da `date/time field value out of range` («valor de fecha fuera de rango»): febrero no tiene día 31.
3. `pg_typeof(columna)` devuelve el tipo de cada columna: `text`, `integer`, `numeric`, `date` y `boolean`.

```

CREATE TABLE precios_texto (p text);

INSERT INTO precios_texto VALUES ('3'), ('20'), ('100');

SELECT p FROM precios_texto ORDER BY p;

SELECT p FROM precios_texto ORDER BY p::numeric;

```

```
psql — base de datos «ensayo»

ensayo=# CREATE TABLE precios_texto (p text);
CREATE TABLE

ensayo=# INSERT INTO precios_texto VALUES ('3'), ('20'), ('100');
INSERT 0 3

ensayo=# SELECT p FROM precios_texto ORDER BY p;
 p
----
100
 20
  3
(3 rows)

ensayo=# SELECT p FROM precios_texto ORDER BY p::numeric;
 p
----
  3
 20
100
(3 rows)
```

Figura 1.5-d. Captura real: ordenar números guardados como texto da un orden sorprendente.

Qué ves en la imagen. Se guardan 3, 20 y 100 **como texto**. `ORDER BY p` (ordenar por `p`) los ordena así: `100`, `20`, `3`, es decir, **alfabéticamente**, no por valor. Al convertirlos a número con `p::numeric` (la doble `::` significa «convertir al tipo»), el orden es el correcto: `3`, `20`, `100`.

Ejemplo sencillo

`nombre` es texto, `stock` es un entero, `precio_eur` es decimal, `alta` es una fecha y `activo` es verdadero o falso. Con eso puedes sumar stocks, calcular totales, ordenar por fecha y filtrar por activos.

Ejemplo real

En una tienda, el `stock` es entero porque no vendes media camiseta; `precio_eur` es decimal porque hay céntimos; `alta` es una fecha para saber desde cuándo existe el producto, y `activo` oculta un producto de la web sin borrarlo.

Profundizando

Teléfonos y códigos postales son texto. Aunque parezcan números, no se suman ni se comparan numéricamente, y pueden llevar ceros iniciales o el signo «+». Regla: si no vas a hacer cálculos con él, guárdalo como texto.

Dinero: `numeric`, no decimales aproximados. Los tipos `real` y `double precision` son aproximados y acumulan errores minúsculos (en el punto 3.1 verás que `0,1 + 0,2` no da exactamente `0,3`). Para dinero, `numeric`.

Fechas. `date` guarda un día; `timestampz` guarda un instante exacto con zona horaria. Para «cuándo ocurrió algo» usa `timestampz`.

Longitud del entero. `integer` admite hasta unos 2.100 millones; para cifras mayores existe `bigint`.

Ejercicio

1. Elige el tipo para cada columna: la edad, el precio, el email, si el usuario está activo y la fecha de alta.
2. ¿Por qué un teléfono (por ejemplo, +34 600 111 222) conviene guardarlo como texto y no como entero?
3. Una columna `precio` guarda texto. ¿Qué problema tendrás para calcular el total de ventas? Compruébalo con una consulta.

Solución

1. Edad: `integer`. Precio: `numeric(10, 2)`. Email: `text`. Activo: `boolean`. Fecha de alta: `date`.
2. Un teléfono no se suma ni se compara numéricamente, y puede llevar el signo «+», espacios o ceros iniciales que un entero perdería.
3. El gestor no puede sumar texto: `SELECT sum(precio) FROM tabla_con_texto;` da un error de tipo. Habría que convertir cada valor con `precio::numeric`, y un valor mal escrito rompería el cálculo.

Error habitual

Guardarlo todo como texto «por si acaso». Funciona al principio, pero impide calcular, ordenar bien y validar.

```
-- Mal: el precio como texto no se puede sumar ni ordenar bien
CREATE TABLE productos_mal (nombre text, precio text);
-- Bien
CREATE TABLE productos_bien (nombre text, precio_eur numeric(10, 2));
```

Buena práctica

Para cada columna pregunta: ¿voy a sumarlo, compararlo, ordenarlo por fecha, o solo mostrarlo? La respuesta marca el tipo.

Comprobación

Para cada columna de tu tabla, escribe qué tipo tiene y por qué. Verifícalo con `\d tabla` o con `pg_typeof`. Inserta a propósito un valor de tipo incorrecto y confirma que se rechaza.

1.6 Valores, nulos e identificadores

¿Qué es?

Un **valor** es el contenido concreto de una celda. Los valores de una fila forman un registro.

NULL es un caso especial: significa «**no hay valor**». El dato no se conoce o no existe. Es importante entender que NULL **no es lo mismo** que:

- el número **0**, que es un valor conocido (cero);
- un **texto vacío** (`' '`), que es un valor conocido (un texto sin letras).

NULL es la ausencia de valor, no un valor.

Un **identificador** es un valor que distingue una fila de todas las demás. El más habitual es una columna `id`: un número que no se repite.

¿Para qué sirve?

NULL permite guardar una fila aunque falte algún dato opcional. El identificador permite señalar una fila concreta sin ambigüedad.

¿Por qué lo necesito?

Hay dos clientes llamados «Luis Pérez». Si alguien dice «cambia el teléfono de Luis Pérez», ¿de cuál habla? Con el identificador es inequívoco: «cambia el teléfono del cliente 2». Los nombres se repiten; los identificadores, no.

Y si Luis no ha dado su teléfono, no puedes inventar uno ni borrar a Luis por eso. Necesitas una forma de decir «el teléfono de Luis no se sabe»: eso es NULL.

¿Cómo funciona?

- Cada columna se define como **obligatoria** (no admite NULL) u **opcional** (admite NULL).
- El `id` es siempre obligatorio y único: el gestor rechaza una segunda fila con el mismo `id`.
- NULL se comporta de forma especial en las comparaciones: «desconocido» no es igual a nada, ni siquiera a otro desconocido. Por eso se pregunta por él con `IS NULL`, no con `= NULL`.

Paso a paso: ver cómo se comporta NULL

1. **Localiza el NULL** en la tabla: el teléfono de Luis.
2. **Pregunta por él con `IS NULL`** y observa que `psql` lo muestra como celda vacía.
3. **Pregunta por él con `= NULL`** y observa que no devuelve nada (el error más común).
4. **Compara NULL con 0 y con el texto vacío** para comprobar que son cosas distintas.
5. **Cuenta con y sin NULL**: `count(*)` cuenta filas; `count(columna)` solo las que tienen valor.
6. **Haz un cálculo con NULL** y observa que el resultado es NULL.

Código y resultado

```
SELECT id, nombre, telefono, telefono IS NULL AS sin_telefono
FROM clientes
ORDER BY id;
```

```
psql — base de datos «tienda»

tienda=# SELECT id, nombre, telefono, telefono IS NULL AS sin_telefono
tienda=# FROM clientes
tienda=# ORDER BY id;
 id | nombre   | telefono | sin_telefono 
----+-----+-----+-----
  1 | Ana García | 600111222 | f
  2 | Luis Pérez |          | t
  3 | Marta Ruiz | 600333444 | f
(3 rows)
```

Figura 1.6-a. Captura real: localizar el NULL de la tabla.

Qué ves en la imagen. La consulta añade una columna calculada, `telefono IS NULL AS sin_telefono`, que vale `t` (verdadero) cuando el teléfono es NULL. Solo Luis tiene `t`. Su celda de teléfono aparece vacía: así muestra `psql` un NULL.

```
SELECT nombre FROM clientes WHERE telefono = NULL;

SELECT nombre FROM clientes WHERE telefono IS NULL;
```

```
psql — base de datos «tienda»

tienda=# SELECT nombre FROM clientes WHERE telefono = NULL;
 nombre
-----
(0 rows)

tienda=# SELECT nombre FROM clientes WHERE telefono IS NULL;
 nombre
-----
Luis Pérez
(1 row)
```

Figura 1.6-b. Captura real: `= NULL` no encuentra nada; `IS NULL` sí.`

Qué ves en la imagen. La primera consulta (`telefono = NULL`) devuelve **0 filas**, aunque Luis tiene el teléfono vacío. Es el error más típico con NULL: comparar «desconocido = desconocido» no da verdadero, da desconocido, y `WHERE` solo deja pasar las filas verdaderas. La segunda (`telefono IS NULL`) devuelve a Luis, como se esperaba.

```

SELECT NULL = NULL AS null_igual_null,
       NULL IS NULL AS es_null,
       0 = 0 AS cero_igual_cero,
       '' = '' AS vacio_igual_vacio;

SELECT count(*) AS todas_las_filas, count(telefono) AS con_telefono FROM
clientes;

SELECT 10 + NULL AS suma_con_null;

```

```

psql — base de datos «tienda»

tienda=# SELECT NULL = NULL AS null_igual_null,
tienda=#       NULL IS NULL AS es_null,
tienda=#       0 = 0 AS cero_igual_cero,
tienda=#       '' = '' AS vacio_igual_vacio;
 null_igual_null | es_null | cero_igual_cero | vacio_igual_vacio
-----+-----+-----+-----
(1 row)

tienda=# SELECT count(*) AS todas_las_filas, count(telefono) AS con_telefono FROM clientes;
 todas_las_filas | con_telefono
-----+-----
(1 row)
      3         |           2

tienda=# SELECT 10 + NULL AS suma_con_null;
 suma_con_null
-----
(1 row)

```

Figura 1.6-c. Captura real: NULL, 0 y texto vacío no son lo mismo; NULL contagia los cálculos.

Qué ves en la imagen.

1. `NULL = NULL` da vacío (NULL), no `t`: no se puede afirmar que dos desconocidos sean iguales. `NULL IS NULL` da `t`. `0 = 0` y `'' = ''` dan `t`: son valores conocidos.
2. `count(*)` devuelve 3 (todas las filas); `count(telefono)` devuelve 2 (solo cuenta las filas donde `telefono` no es NULL).
3. `10 + NULL` da NULL: cualquier cálculo con un dato desconocido da un resultado desconocido.

tienda · clientes			
id	nombre	email	telefono
1	Ana García	ana@ejemplo.com	600111222
2	Luis Pérez	luis@ejemplo.com	NULL
3	Marta Ruiz	marta@ejemplo.com	600333444

Tres casos que parecen iguales

`NULL` — no se sabe / no hay valor

`0` — valor conocido: cero

`''` — valor conocido: texto sin letras

Figura 1.6-d. Ilustración: Luis tiene el teléfono en NULL; a la derecha, tres casos que parecen iguales.

Qué ves en la imagen. A la izquierda, la tabla con el NULL de Luis. A la derecha, la diferencia entre NULL (no se sabe), 0 (valor conocido: cero) y texto vacío (valor conocido: un texto sin letras).

Ejemplo sencillo

Luis no ha dado su teléfono. En su fila, la celda `telefono` es NULL. Su fila sigue siendo válida: tiene `id`, `nombre` y `email`.

Ejemplo real

Una tabla `suscripciones` tiene una columna `fecha_baja`. Mientras el cliente sigue activo, `fecha_baja` es NULL. Cuando se da de baja, se rellena con la fecha. Para saber quién está activo basta con buscar las filas donde `fecha_baja IS NULL`.

Profundizando

Lógica de tres valores. En SQL una condición puede ser verdadera, falsa o **desconocida**. `WHERE` solo deja pasar lo verdadero. Por eso `WHERE telefono <> '600111222'` tampoco devuelve a Luis: no se sabe si su teléfono es distinto o no.

Funciones para tratar NULL. `COALESCE(telefono, 'sin teléfono')` sustituye un NULL por un valor; `IS NOT NULL` pregunta lo contrario de `IS NULL`. Se estudian en la Parte 5.

¿Obligatorio u opcional? Haz obligatorio (`NOT NULL`) lo que la fila necesita para tener sentido. Cuantos más NULL, más casos especiales en cada consulta.

Ejercicio

1. Un producto no tiene descuento. ¿Pondrías 0, un texto vacío o NULL? Razona.
2. Escribe la consulta que devuelve los clientes que sí tienen teléfono.
3. Escribe una consulta que devuelva cuántos clientes no tienen teléfono (usa `count(*)` y `IS NULL`).

Solución

1. Depende del significado. Si «sin descuento» equivale a un 0 %, usa 0 (valor conocido). Si el descuento aún no se ha decidido, usa NULL. Lo importante es decidirlo una vez y ser coherente.
2. `SELECT nombre, telefono FROM clientes WHERE telefono IS NOT NULL;` devuelve Ana y Marta.
3. `SELECT count(*) FROM clientes WHERE telefono IS NULL;` devuelve 1.

Error habitual

Escribir `= NULL` en lugar de `IS NULL`, y usar valores inventados como «-1», «sin dato» o «N/A» en lugar de NULL.

```
-- Mal: nunca devuelve filas
SELECT * FROM clientes WHERE telefono = NULL;
-- Bien
SELECT * FROM clientes WHERE telefono IS NULL;
```

Buena práctica

Haz obligatorias (**NOT NULL**) las columnas sin las cuales la fila no tiene sentido. Usa NULL solo cuando la ausencia del dato sea legítima.

Comprobación

Para cada columna decide «obligatoria» u «opcional». Para cada opcional, escribe qué significa que esté vacía. Comprueba con **IS NULL** que tus consultas encuentran los vacíos.

1.7 Claves: primaria y extranjera

¿Qué es?

Una **clave** es una columna (o un grupo de columnas) que sirve para identificar o enlazar filas. Hay dos tipos esenciales:

- La **clave primaria** (en inglés, *primary key*, **PK**) es la columna que identifica de forma única cada fila de una tabla. En `clientes` es el `id`. Nunca se repite y nunca está vacía.
- La **clave extranjera** (en inglés, *foreign key*, **FK**) es una columna que guarda la clave primaria de **otra** tabla, para enlazar las dos. En `pedidos`, `cliente_id` guarda el `id` del cliente que hizo el pedido.

¿Para qué sirve?

La clave primaria permite señalar una fila concreta sin duda posible. La clave extranjera permite **relacionar tablas sin repetir datos**: en lugar de copiar el nombre y el teléfono de Ana en cada pedido, el pedido solo guarda el número 1, que es el `id` de Ana.

¿Por qué lo necesito?

Sin claves no puedes enlazar un pedido con su cliente ni garantizar que no haya dos filas idénticas. Y si copiaras los datos del cliente en cada pedido, volverías al problema del punto 1.2: datos repetidos que se contradicen.

¿Cómo funciona?

El gestor vigila las claves por ti:

1. **Con la clave primaria**: rechaza cualquier fila nueva cuyo `id` ya exista.
2. **Con la clave extranjera**: rechaza un pedido cuyo `cliente_id` no corresponda a ningún cliente existente. No puede haber un pedido «de nadie».
3. **Al borrar**: impide borrar un cliente que aún tiene pedidos, para no dejar pedidos huérfanos (es configurable; se estudia en la Parte 3).

Primero, sin código: enlaza las tablas a mano

Las relaciones también se entienden mejor **primero sobre papel**. El truco es uno solo: **para enlazar dos tablas, en una de ellas escribes el número (id) de la fila de la otra**.

1. Cada cliente lleva un **número** que lo identifica (el `id`).
2. Dibuja la tabla de pedidos y añade una **columna de enlace** (`cliente_id`), todavía vacía.
3. Por cada pedido pregunta «¿de quién es?» y escribe **el número del cliente**, no su nombre.
4. **Comprueba** leyendo en los dos sentidos: de pedido a cliente y de cliente a pedidos.

1 Cada cliente necesita un número que lo identifique

clientes	
id	nombre
1	Ana García
2	Luis Pérez
3	Marta Ruiz

Ese número (id) no se repite nunca. Será la clave primaria.

2 Crea la tabla de pedidos con una columna para enlazar

clientes		pedidos		
id	nombre	id	cliente_id	total
1	Ana García	101	?	45.90
2	Luis Pérez	102	?	12.00
3	Marta Ruiz	103	?	80.50

La columna cliente_id está vacía por ahora (?). Guardará el número de cliente al que pertenece cada pedido.

3 Pregunta por cada pedido: ¿de quién es? y escribe SU NÚMERO

clientes		pedidos		
id	nombre	id	cliente_id	total
1	Ana García	101	1	45.90
2	Luis Pérez	102	3	12.00
3	Marta Ruiz	103	1	80.50

El pedido 101 es de Ana → 1. El 102 es de Marta → 3. El 103 es de Ana → 1. No se copia el nombre, solo el número.

4 Comprueba leyendo en los dos sentidos

De pedido a cliente: pedido 102 → cliente_id 3 → **Marta Ruiz**.

De cliente a pedidos: Ana (id 1) → los pedidos cuyo cliente_id es 1 → **101 y 103**. Luis (id 2) → ninguno.

Si un cliente_id no existe en la tabla clientes, el enlace está roto. Eso es lo que PostgreSQL vigilará por ti (con código).

Figura 1.7-a. Sin código: enlazar PEDIDOS con CLIENTES a mano, en cuatro pasos.

Qué ves en la imagen. Paso 1: los clientes con su número. Paso 2: la tabla de pedidos con la columna `cliente_id` por rellenar (los `?`). Paso 3: cada pedido recibe el número de su cliente; los colores muestran qué pedido es de quién (Ana en azul, Marta en naranja). Paso 4: la comprobación en los dos sentidos. Si un número de `cliente_id` no apareciera en la tabla `clientes`, el enlace estaría roto: eso es lo que PostgreSQL vigilará por ti en el siguiente apartado.

Paso a paso: ahora con código — crear `pedidos` enlazada a `clientes`

1. Comprueba que `clientes` ya existe y tiene clave primaria (`id`).
2. Crea `pedidos` con una columna `cliente_id` declarada con `REFERENCES clientes (id)`.
3. Inserta pedidos válidos (de clientes que existen) y léelos.
4. Intenta romper la clave primaria: inserta un cliente con un `id` repetido.

5. Intenta romper la clave extranjera: inserta un pedido de un cliente inexistente.
6. Intenta borrar un cliente con pedidos.
7. Une las dos tablas con `JOIN` para ver el enlace en acción.

Código y resultado

```
CREATE TABLE pedidos (
  id          integer PRIMARY KEY,
  cliente_id integer NOT NULL REFERENCES clientes (id),
  fecha       date,
  total       numeric(10, 2)
);

INSERT INTO pedidos VALUES
  (101, 1, '2026-03-02', 45.90),
  (102, 3, '2026-03-05', 12.00),
  (103, 1, '2026-03-09', 80.50);

SELECT * FROM pedidos;
```



```
psql — base de datos «tienda»

tienda=# CREATE TABLE pedidos (
tienda-#   id          integer PRIMARY KEY,
tienda-#   cliente_id integer NOT NULL REFERENCES clientes (id),
tienda-#   fecha       date,
tienda-#   total       numeric(10, 2)
tienda-# );
CREATE TABLE

tienda=# INSERT INTO pedidos VALUES
tienda-#   (101, 1, '2026-03-02', 45.90),
tienda-#   (102, 3, '2026-03-05', 12.00),
tienda-#   (103, 1, '2026-03-09', 80.50);
INSERT 0 3

tienda=# SELECT * FROM pedidos;
 id | cliente_id | fecha       | total
-----+-----+-----+-----
 101 |          1 | 2026-03-02 | 45.90
 102 |          3 | 2026-03-05 | 12.00
 103 |          1 | 2026-03-09 | 80.50
(3 rows)
```

Figura 1.7-b. Captura real: la tabla `pedidos`, con su clave primaria y su clave extranjera hacia `clientes`.

Qué ves en la imagen.

1. `id integer PRIMARY KEY`: la clave primaria de `pedidos`.
2. `cliente_id integer NOT NULL REFERENCES clientes (id)`: esta es la clave extranjera. Se lee: «`cliente_id` es un entero obligatorio y su valor debe existir en la columna `id` de `clientes`».
3. Se insertan tres pedidos: el 101 y el 103 son del cliente 1 (Ana) y el 102 del cliente 3 (Marta). Luis (cliente 2) no tiene pedidos.

```
INSERT INTO clientes VALUES (1, 'Otro Cliente', 'otro@ejemplo.com', NULL);

INSERT INTO pedidos VALUES (104, 9, '2026-03-12', 20.00);

DELETE FROM clientes WHERE id = 1;
```



```
psql — base de datos «tienda»

tienda=# INSERT INTO clientes VALUES (1, 'Otro Cliente', 'otro@ejemplo.com', NULL);
ERROR: duplicate key value violates unique constraint "clientes_pkey"
DETAIL: Key (id)=(1) already exists.

tienda=# INSERT INTO pedidos VALUES (104, 9, '2026-03-12', 20.00);
ERROR: insert or update on table "pedidos" violates foreign key constraint "pedidos_cliente_id_fkey"
DETAIL: Key (cliente_id)=(9) is not present in table "clientes".

tienda=# DELETE FROM clientes WHERE id = 1;
ERROR: update or delete on table "clientes" violates foreign key constraint "pedidos_cliente_id_fkey" on table "pedidos"
DETAIL: Key (id)=(1) is still referenced from table "pedidos".
```

Figura 1.7-c. Captura real: tres operaciones que el gestor rechaza para proteger los datos.

Qué ves en la imagen.

1. Insertar un cliente con el `id` 1, que ya existe: `duplicate key value violates unique constraint "clientes_pkey"`, con el detalle `Key (id)=(1) already exists` («la clave `id=1` ya existe»).
2. Insertar un pedido para el cliente 9, que no existe: `violates foreign key constraint "pedidos_cliente_id_fkey"`, con `Key (cliente_id)=(9) is not present in table "clientes"` («la clave 9 no está en la tabla `clientes` »).
3. Borrar al cliente 1, que todavía tiene pedidos: `violates foreign key constraint ... on table "pedidos"`, con `Key (id)=(1) is still referenced from table "pedidos"` («la clave sigue referenciada»).

En los tres casos **no se guarda ni se borra nada**: la base de datos se queda como estaba.

```
SELECT p.id AS pedido, p.total, c.nombre AS cliente
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
ORDER BY p.id;
```

```

psql — base de datos «tienda»

tienda=# SELECT p.id AS pedido, p.total, c.nombre AS cliente
tienda=# FROM pedidos p
tienda=# JOIN clientes c ON c.id = p.cliente_id
tienda=# ORDER BY p.id;
 pedido | total | cliente
-----+-----+-----
    101 | 45.90 | Ana García
    102 | 12.00 | Marta Ruiz
    103 | 80.50 | Ana García
(3 rows)

```

Figura 1.7-d. Captura real: unir las dos tablas para ver cada pedido con su cliente.

Qué ves en la imagen. La consulta usa **JOIN** («unir») para combinar cada pedido con la fila del cliente cuyo **id** coincide con su **cliente_id** (**ON c.id = p.cliente_id**). El resultado muestra el nombre del cliente junto a cada pedido: Ana en el 101 y el 103, Marta en el 102. Luis no aparece, porque no tiene pedidos. La letra **p** y la **c** son **alias** (apodos cortos) de las tablas **pedidos** y **clientes**.



Figura 1.7-e. Ilustración: claves primaria (PK) y extranjera (FK) enlazando PEDIDOS con CLIENTES.

Qué ves en la imagen. Cada pedido tiene el mismo color que su cliente: Ana (azul) tiene los pedidos 101 y 103, Marta (naranja) el 102 y Luis, sin pedidos, no aparece en ellos. El enlace es el número de **cliente_id**.

Ejemplo sencillo

El pedido 101 tiene **cliente_id** = 1. Buscas el 1 en la tabla de clientes: es Ana García.

Ejemplo real

En una tienda con 20.000 pedidos, la tabla **pedidos** solo guarda un número para identificar al cliente. Cuando Ana cambia su teléfono, se modifica una sola fila en **clientes** y todos sus pedidos «ven» el dato nuevo, porque nunca lo copiaron.

Profundizando

Qué hace buena a una clave primaria. Debe ser única, no vacía y **estable** (que nunca cambie). Por eso un número sin significado propio es mejor que un email o un nombre, que pueden cambiar. En el punto 3.4 veremos las opciones (identidad automática, UUID y claves naturales).

Claves compuestas. Una clave primaria puede estar formada por varias columnas a la vez (por ejemplo, pedido y producto). Se estudia en el punto 3.3.

Una clave extranjera puede apuntar a una tabla consigo misma (un empleado y su jefe). Se estudia en el punto 3.2.

Ejercicio

1. En la figura 1.7-e, ¿de quién es el pedido 102? ¿Cuántos pedidos tiene Ana?
2. Luis quiere hacer un pedido nuevo. Escribe el `INSERT` (con `id` 104, fecha 2026-03-12 y total 20.00).
3. ¿Qué pasaría si intentas guardar un cliente con `id` = 2, que ya existe? ¿Y borrar a Luis, que no tiene pedidos?

Solución

1. El pedido 102 es de Marta (`cliente_id` = 3). Ana tiene 2 pedidos: el 101 y el 103.
2. `INSERT INTO pedidos VALUES (104, 2, '2026-03-12', 20.00);` El `cliente_id` es 2, el `id` de Luis. Su nombre no se escribe en el pedido.
3. Guardar un `id` repetido sería rechazado por la clave primaria. Borrar a Luis sí se permitiría, porque ningún pedido apunta a él.

Error habitual

Usar el nombre o el email como clave primaria. Pueden cambiar o repetirse, y si cambian hay que actualizarlos en todas las tablas que los usan.

```
-- Mal: dos Luis Pérez harían imposible esta tabla
CREATE TABLE clientes_mal (nombre text PRIMARY KEY, email text);
-- Bien: un identificador que nunca cambia
CREATE TABLE clientes_bien (id integer PRIMARY KEY, nombre text, email text);
```

Buena práctica

Dale a cada tabla una clave primaria, y usa claves extranjeras para toda relación entre tablas. Así el gestor protege la coherencia de tus datos sin que tengas que vigilarla a mano.

Comprobación

Para cada tabla pregunta: ¿qué columna identifica cada fila? ¿Con qué otras tablas se enlaza y mediante qué columna? Comprueba con `\d tabla` que aparecen la clave primaria y las claves extranjeras esperadas.

1.8 Índices

¿Qué es?

Un **índice** es una estructura que la base de datos mantiene aparte de la tabla para encontrar filas rápidamente. Funciona igual que el índice alfabético de un libro: en lugar de leer las 400 páginas para encontrar la palabra «bosque», vas al índice, ves en qué página está y saltas directamente a ella.

¿Para qué sirve?

Acelera las búsquedas por una columna, por ejemplo buscar un cliente por su email o los pedidos de un cliente por `cliente_id`.

¿Por qué lo necesito?

Con 10 filas, da igual cómo se busque. Con millones, recorrer la tabla fila a fila para encontrar un email es lento, y esa lentitud se repite cada vez que alguien inicia sesión. Con un índice, la búsqueda es casi inmediata. En proyectos grandes, los índices son una de las herramientas que más rendimiento ganan.

¿Cómo funciona?

Sin índice, el gestor hace una **lectura completa** de la tabla (en inglés, *sequential scan*): mira fila por fila hasta encontrar la que busca. Con índice:

1. El gestor guarda aparte los valores de la columna **ordenados**, cada uno con la posición de su fila.
2. Para buscar un valor, lo localiza en esa lista ordenada (más rápido, porque al estar ordenada no hay que mirarla toda).
3. Salta directamente a la fila indicada.

Es una simplificación; en las Partes 6 y 18 veremos los tipos de índice y cómo comprobar si se usan. Los índices tienen un coste: ocupan espacio y hay que actualizarlos cada vez que se inserta, modifica o borra una fila. Por eso no se crean en todas las columnas.

Paso a paso: medir la diferencia con y sin índice

1. **Crea una tabla grande**: 300.000 clientes generados automáticamente.
2. **Actualiza las estadísticas** con `ANALYZE` (el gestor las usa para elegir cómo buscar).
3. **Pide el plan de una búsqueda por email** con `EXPLAIN ANALYZE`: el gestor ejecuta la consulta y te cuenta cómo la resolvió y cuánto tardó.
4. **Crea un índice** sobre la columna `email`.
5. **Repite la medición** y compara los tiempos.

Código y resultado

```
CREATE TABLE clientes_grandes AS
SELECT g AS id, 'cliente' || g || '@ejemplo.com' AS email
FROM generate_series(1, 300000) AS g;

ANALYZE clientes_grandes;

EXPLAIN ANALYZE
SELECT * FROM clientes_grandes
WHERE email = 'cliente250000@ejemplo.com';
```



```
psql — base de datos «ensayo»

ensayo=# CREATE TABLE clientes_grandes AS
ensayo=# SELECT g AS id, 'cliente' || g || '@ejemplo.com' AS email
ensayo=# FROM generate_series(1, 300000) AS g;
ensayo=# SELECT 300000

ensayo=# ANALYZE clientes_grandes;
ANALYZE

ensayo=# EXPLAIN ANALYZE
ensayo=# SELECT * FROM clientes_grandes
ensayo=# WHERE email = 'cliente250000@ejemplo.com';

QUERY PLAN
-----
Gather  (cost=1000.00..5445.98 rows=1 width=29) (actual time=11.836..16.475 rows=1 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  -> Parallel Seq Scan on clientes_grandes  (cost=0.00..4445.88 rows=1 width=29) (actual time=10.256..11.263 rows=0 loops=2)
        Filter: (email = 'cliente250000@ejemplo.com'::text)
        Rows Removed by Filter: 150000
Planning Time: 0.258 ms
Execution Time: 16.525 ms
(8 rows)
```

Figura 1.8-a. Captura real: buscar un email en 300.000 filas sin índice.

Qué ves en la imagen, orden por orden.

1. `CREATE TABLE clientes_grandes AS SELECT ... FROM generate_series(1, 300000)`: crea una tabla a partir del resultado de una consulta. `generate_series(1, 300000)` genera los números del 1 al 300.000; de cada uno sale un `id` y un email como `cliente250000@ejemplo.com`. La respuesta `SELECT 300000` indica que se han creado 300.000 filas.
2. `ANALYZE` recoge estadísticas de la tabla.
3. `EXPLAIN ANALYZE SELECT ... WHERE email = '...'` ejecuta la búsqueda y muestra el **plan**. Lo importante: la línea `Parallel Seq Scan on clientes_grandes` significa que el gestor ha leído **toda la tabla** (con ayuda de más de un proceso). La línea `Rows Removed by Filter` cuenta las filas que ha tenido que descartar una a una. Y al final, `Execution Time: 16.525 ms`: el tiempo real de esta búsqueda.

```
CREATE INDEX idx_grandes_email ON clientes_grandes (email);

ANALYZE clientes_grandes;

EXPLAIN ANALYZE
SELECT * FROM clientes_grandes
WHERE email = 'cliente250000@ejemplo.com';
```



```
psql — base de datos «ensayo»

ensayo=# CREATE INDEX idx_grandes_email ON clientes_grandes (email);
CREATE INDEX

ensayo=# ANALYZE clientes_grandes;
ANALYZE

ensayo=# EXPLAIN ANALYZE
ensayo-# SELECT * FROM clientes_grandes
ensayo-# WHERE email = 'cliente250000@ejemplo.com';

QUERY PLAN
-----
Index Scan using idx_grandes_email on clientes_grandes  (cost=0.42..8.44 rows=1 width=29) (actual time=0.033..0.034 rows=1 loops=1)
  Index Cond: (email = 'cliente250000@ejemplo.com'::text)
Planning Time: 0.335 ms
Execution Time: 0.068 ms
(4 rows)
```

Figura 1.8-b. Captura real: la misma búsqueda después de crear un índice sobre `email`.

Qué ves en la imagen.

1. `CREATE INDEX idx_grandes_email ON clientes_grandes (email)` crea el índice (el nombre `idx_grandes_email` lo elegimos nosotros).
2. La misma consulta ahora muestra **Index Scan using idx_grandes_email**: el gestor ha usado el índice y ha saltado directamente a la fila. Ya no hay **Rows Removed by Filter**. El **Execution Time** baja a **0.068 ms**.

En esta ejecución, la búsqueda con índice ha sido unas **243 veces más rápida**. Los tiempos exactos dependen de tu ordenador y cambian en cada ejecución, pero la diferencia de orden de magnitud se mantiene, y crece con el tamaño de la tabla.

Índice de un libro		Índice de la columna email	
Palabra	Página	email (ordenado)	fila
bosque	212	ana@ejemplo.com	1
cielo	45	luis@ejemplo.com	2
río	133	marta@ejemplo.com	3
sol	301		

Ordenado: vas directo a la página

Buscas marta@... → fila 3, sin recorrer todo

Figura 1.8-c. Ilustración: el índice de un libro y el índice de una columna funcionan igual.

Qué ves en la imagen. A la izquierda, el índice de un libro: palabras ordenadas con su página. A la derecha, el índice de la columna `email`: emails ordenados con el número de fila donde están. Ambos sirven para saltar directamente sin leer todo.

Ejemplo sencillo

Buscar «marta@ejemplo.com» en una tabla de 3 filas: no hace falta índice. En una de 300.000 filas, sí.

Ejemplo real

La tabla `pedidos` tiene 5 millones de filas. La web muestra «Mis pedidos» y busca los de un cliente por `cliente_id`. Con un índice sobre `cliente_id`, el gestor localiza esos pedidos sin recorrer los 5 millones.

Profundizando

Índices automáticos. PostgreSQL crea un índice por cada clave primaria y por cada restricción `UNIQUE` (lo viste en `\d clientes`). Para las claves extranjeras no lo hace: hay que crearlo tú (punto 3.3).

Cuándo el gestor NO usa un índice. En tablas pequeñas, o cuando la consulta devuelve gran parte de las filas, leer la tabla entera es más barato que saltar a muchas posiciones. El gestor decide por sí mismo, con las estadísticas.

Coste de escritura. Cada índice hay que actualizarlo en cada `INSERT`, `UPDATE` y `DELETE`. Muchos índices ralentizan las escrituras y consumen disco.

Tipos de índice. Aquí hemos usado el tipo habitual (B-tree). Existen otros para usos especiales (texto completo, JSON, geografía); se ven en la Parte 6.

Ejercicio

1. ¿Qué columna de `clientes` indexarías si tu aplicación busca clientes por correo?
2. ¿Qué columna de `pedidos` indexarías para mostrar «los pedidos de este cliente»?
3. Escribe la sentencia que crea un índice en esa columna de `pedidos`.
4. ¿Por qué no es buena idea crear un índice en cada columna de todas las tablas?

Solución

1. La columna `email`.
2. La columna `cliente_id`, que es por la que se busca.
3. `CREATE INDEX idx_pedidos_cliente ON pedidos (cliente_id);`
4. Cada índice ocupa espacio y hay que actualizarlo en cada inserción, modificación o borrado. Demasiados índices ralentizan las escrituras y consumen disco sin aportar nada si nadie busca por esas columnas.

Error habitual

Poner índices a todo «por si acaso», o pensar que un índice sirve para guardar más datos o para ordenar la tabla visualmente. Un índice solo ayuda a **encontrar**.

```
-- Mal: un índice sobre una columna que casi nunca se consulta  
CREATE INDEX idx_clientes_telefono ON clientes (telefono);
```

Buena práctica

Indexa las columnas por las que buscas, enlazas u ordenas con frecuencia, y mide antes y después con `EXPLAIN ANALYZE`. Una columna que casi nunca se consulta no necesita índice.

Comprobación

Ejecuta tu consulta con `EXPLAIN ANALYZE` antes y después de crear el índice. Debes ver cómo pasa de `Seq Scan` a `Index Scan` y cómo baja el tiempo. Si no cambia, el índice no está ayudando.

1.9 Consultas

¿Qué es?

Una **consulta** es una petición que se le hace a la base de datos. Cuando la petición es una pregunta («dame los clientes»), la base de datos responde con filas. Las consultas se escriben en **SQL** (se pronuncia «ese-cu-ele»), el lenguaje de las bases de datos relacionales. Aprenderás SQL desde cero en la Parte 5.

Una consulta puede pedir datos, pero también crearlos, modificarlos o borrarlos. En el uso diario, casi todas son preguntas.

¿Para qué sirve?

Para sacar información de los datos guardados: listar, buscar, filtrar, ordenar, contar, sumar. Y para cambiar los datos de forma controlada.

¿Por qué lo necesito?

Guardar datos no sirve de nada si no puedes recuperarlos. Cada pantalla de una aplicación (la lista de pedidos, el perfil, el buscador) ejecuta una o varias consultas por detrás.

¿Cómo funciona?

1. Escribes (o la aplicación escribe) la consulta.
2. La envías al gestor.
3. El gestor la ejecuta y construye una **tabla de resultados**, con cero, una o muchas filas.
4. El resultado se devuelve a quien preguntó.

El resultado no es una tabla guardada: es una tabla temporal, creada para esa pregunta. La estructura básica de una consulta de lectura es siempre la misma:

Parte	Significado	Ejemplo
<code>SELECT</code>	Qué columnas quiero	<code>SELECT nombre, email</code>
<code>FROM</code>	De qué tabla	<code>FROM clientes</code>
<code>WHERE</code>	Qué filas quiero (filtro)	<code>WHERE total > 40</code>
<code>ORDER BY</code>	En qué orden	<code>ORDER BY total DESC</code>
<code>LIMIT</code>	Cuántas filas como máximo	<code>LIMIT 1</code>

Primero, sin código: contesta tú mismo con el dedo

Una consulta es una pregunta sobre una tabla. Antes de escribirla, **contéstala a mano**:

1. Tienes la tabla de pedidos delante: 101 (45,90 €), 102 (12,00 €) y 103 (80,50 €).

2. Pregunta: «¿qué pedidos pasan de 40 euros?». Recorre las filas **con el dedo**, una por una, y tacha las que no cumplen: el 102 (12,00) queda fuera.
3. Respuesta: **101 y 103**. Si quieres «el más caro», compara los totales y te quedas con el 103.
4. Esto es exactamente lo que hará el gestor, pero con millones de filas y en milisegundos.

Si sabes responder así, escribir la consulta es solo traducir tu pregunta a SQL.

Paso a paso: ahora con código — formular, escribir y leer una consulta

1. **Formula la pregunta en español:** «dame el nombre y el email de todos los clientes».
2. **Decide qué columnas necesitas** (no todas) y de qué tabla salen.
3. **Escribe la consulta** y termínala con `;`.
4. **Antes de ejecutarla, calcula cuántas filas esperas.**
5. **Ejecútala y compara** con lo esperado.
6. **Añade filtros, orden y límite** poco a poco, comprobando cada cambio.

Código y resultado

```
SELECT * FROM clientes;
```

```
SELECT nombre, email FROM clientes;
```

The screenshot shows a terminal window titled "psql — base de datos «tienda»". It contains two SQL queries and their results:

```
tienda=# SELECT * FROM clientes;
 id | nombre | email | telefono
-----+-----+-----+-----
  1 | Ana García | ana@ejemplo.com | 600111222
  2 | Luis Pérez | luis@ejemplo.com | 
  3 | Marta Ruiz | marta@ejemplo.com | 600333444
(3 rows)
```

```
tienda=# SELECT nombre, email FROM clientes;
 nombre | email
-----+-----
 Ana García | ana@ejemplo.com
 Luis Pérez | luis@ejemplo.com
 Marta Ruiz | marta@ejemplo.com
(3 rows)
```

Figura 1.9-a. Captura real: una consulta que pide todas las columnas y otra que pide solo dos.

Qué ves en la imagen.

1. `SELECT * FROM clientes` devuelve **todas** las columnas y **todas** las filas (3 filas, 4 columnas).
2. `SELECT nombre, email FROM clientes` devuelve solo las columnas pedidas: `nombre` y `email`. Las columnas `id` y `telefono` no aparecen porque no se pidieron. Las filas siguen siendo 3.

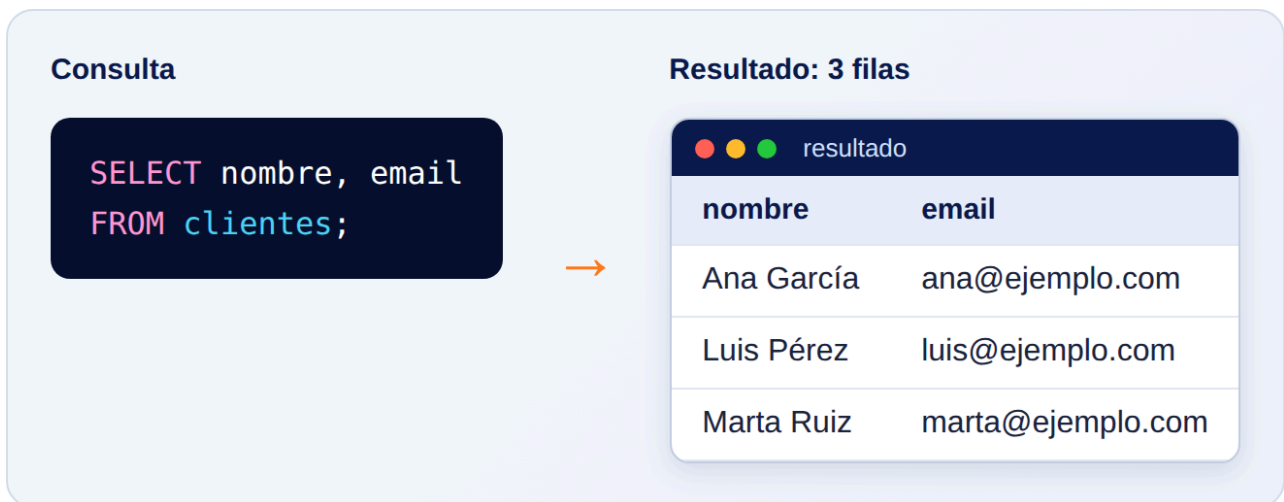


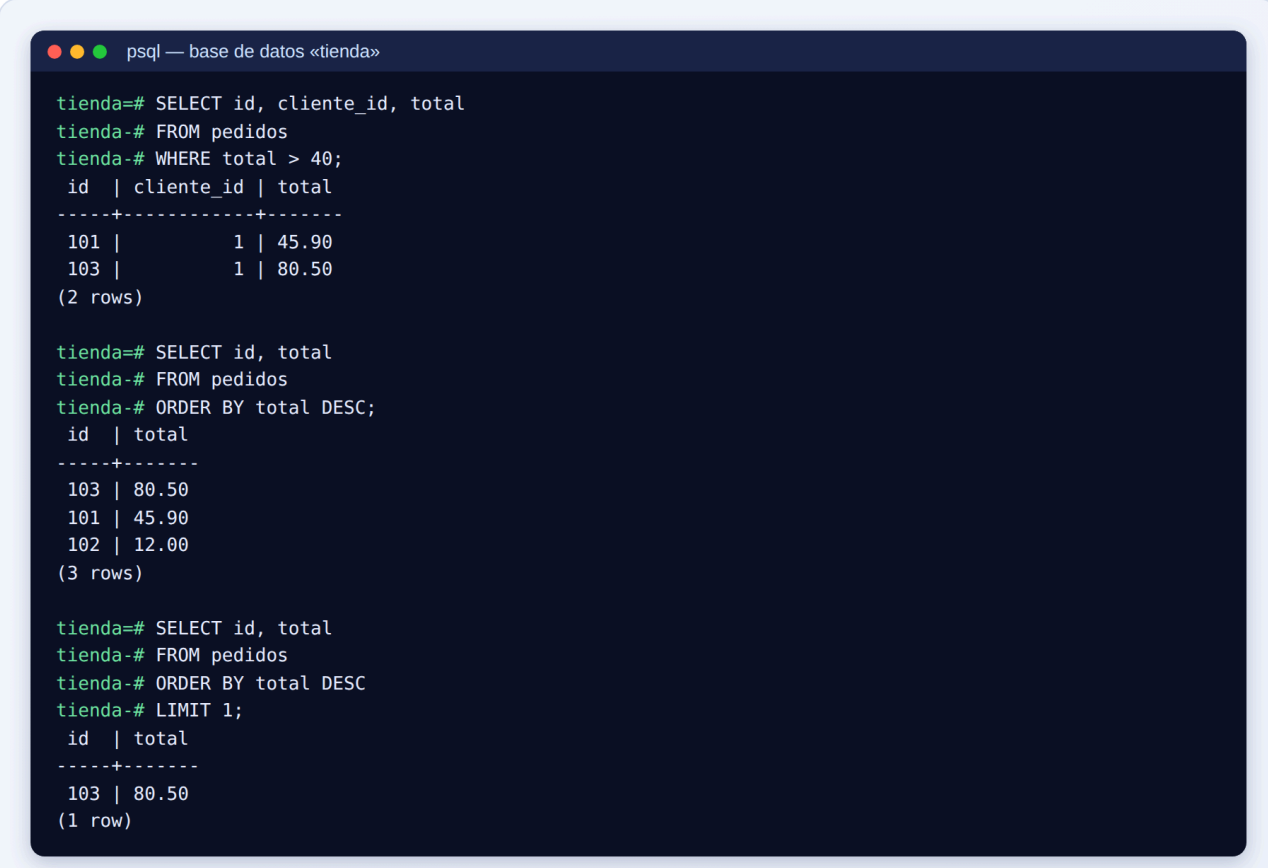
Figura 1.9-b. Ilustración: una consulta y su resultado.

Qué ves en la imagen. A la izquierda, la consulta; a la derecha, la tabla de resultados que devuelve.

```
SELECT id, cliente_id, total
FROM pedidos
WHERE total > 40;
```

```
SELECT id, total
FROM pedidos
ORDER BY total DESC;
```

```
SELECT id, total
FROM pedidos
ORDER BY total DESC
LIMIT 1;
```



```
psql — base de datos «tienda»

tienda=# SELECT id, cliente_id, total
tienda=# FROM pedidos
tienda=# WHERE total > 40;
 id | cliente_id | total 
-----+-----+-----
 101 |          1 | 45.90
 103 |          1 | 80.50
(2 rows)

tienda=# SELECT id, total
tienda=# FROM pedidos
tienda=# ORDER BY total DESC;
 id | total 
-----+-----
 103 | 80.50
 101 | 45.90
 102 | 12.00
(3 rows)

tienda=# SELECT id, total
tienda=# FROM pedidos
tienda=# ORDER BY total DESC
tienda=# LIMIT 1;
 id | total 
-----+-----
 103 | 80.50
(1 row)
```

Figura 1.9-c. Captura real: filtrar con `WHERE`, ordenar con `ORDER BY` y limitar con `LIMIT`.

Qué ves en la imagen.

1. `WHERE total > 40` deja pasar solo los pedidos de más de 40 euros: el 101 (45,90) y el 103 (80,50). El 102 (12,00) no cumple y no aparece.
2. `ORDER BY total DESC` ordena de mayor a menor (`DESC` significa descendente; sin él, el orden es ascendente): 80,50, 45,90 y 12,00.
3. `ORDER BY total DESC LIMIT 1` ordena y se queda solo con la primera fila: el pedido más caro, el 103.

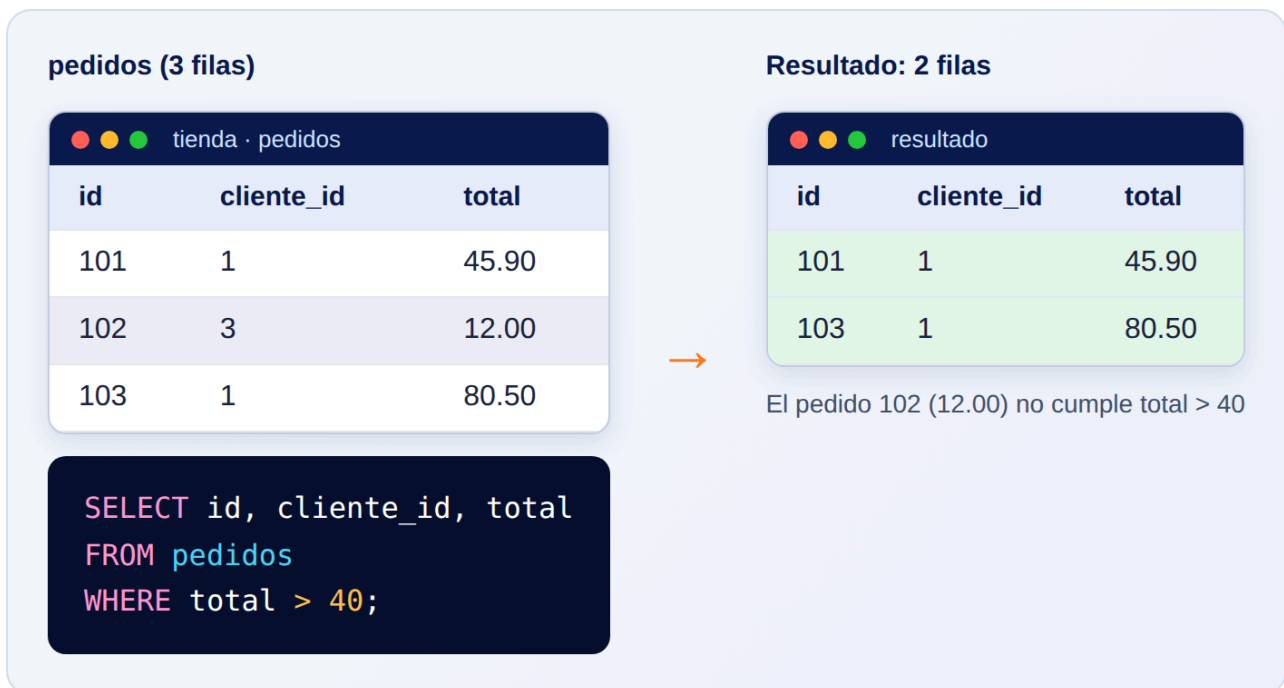


Figura 1.9-d. Ilustración: una consulta con filtro.

Qué ves en la imagen. A la izquierda, los tres pedidos completos; a la derecha, el resultado con los dos que cumplen la condición.

```
SELECT count(*) AS pedidos,
       sum(total) AS facturado,
       round(avg(total), 2) AS media
FROM pedidos;
```

```
psql — base de datos «tienda»

tienda=# SELECT count(*) AS pedidos,
tienda=#       sum(total) AS facturado,
tienda=#       round(avg(total), 2) AS media
tienda=# FROM pedidos;
 pedidos | facturado | media 
-----+-----+-----
       3 |    138.40 |  46.13 
(1 row)
```

Figura 1.9-e. Captura real: una consulta que resume en lugar de listar.

Qué ves en la imagen. Las funciones `count`, `sum` y `avg` (contar, sumar y media) resumen muchas filas en una sola. Hay 3 pedidos que suman 138,40 euros, con una media de 46,13 (`round(..., 2)` redondea a dos decimales). Verás estas funciones en detalle en la Parte 5.

Ejemplo sencillo

«Dame el nombre y el email de todos los clientes»: `SELECT nombre, email FROM clientes;`

Ejemplo real

Al abrir «Mis pedidos» en una tienda, la aplicación pregunta a la base de datos por los pedidos de tu usuario y con las filas devueltas dibuja la lista en pantalla.

Profundizando

Una consulta no cambia los datos. `SELECT` solo lee. Las órdenes que modifican son `INSERT`, `UPDATE` y `DELETE` (Parte 5).

El orden de escritura no es el orden de ejecución. Se escribe `SELECT ... FROM ... WHERE ...`, pero el gestor primero resuelve `FROM`, después filtra con `WHERE` y al final elige las columnas del `SELECT`. Lo veremos en la Parte 5.

Pedir solo lo necesario. Con tablas grandes, `SELECT *` trae datos que quizá no uses. Pide solo las columnas y las filas que necesitas: es más rápido y expone menos datos.

Ejercicio

1. Con los pedidos de la tienda, formula en español «los pedidos de Ana» y escribe la consulta (el `id` de Ana es 1).
2. ¿Cuántas filas devolvería `SELECT * FROM pedidos WHERE total < 20;` ?
3. Escribe una consulta que devuelva el nombre del cliente cuyo `id` es 3.

Solución

1. «Dame los pedidos cuyo `cliente_id` sea 1»: `SELECT * FROM pedidos WHERE cliente_id = 1;` devuelve los pedidos 101 y 103.
2. 1 fila: el pedido 102 (12,00).
3. `SELECT nombre FROM clientes WHERE id = 3;` devuelve `Marta Ruiz`.

Error habitual

Pedir siempre todos los datos de todas las filas «por si acaso», y escribir un `UPDATE` o `DELETE` sin `WHERE` (afectaría a **todas** las filas).

```
-- Mal: trae todo aunque solo necesites un nombre
SELECT * FROM clientes;
-- Bien
SELECT nombre FROM clientes WHERE id = 3;
```

Buena práctica

Formula primero la pregunta en español; después escribe la consulta. Y antes de ejecutarla, calcula cuántas filas esperas obtener: así detectarás los errores de lógica.

Comprobación

Antes de ejecutar una consulta, ¿sabrías decir qué columnas devolverá y cuántas filas? Si no, aún no has formulado bien la pregunta. Después de ejecutarla, comprueba que el resultado coincide con tu previsión.

1.10 El entorno: cliente, servidor, API, VPS y nube

¿Qué es?

En este manual aparecerán cinco palabras del entorno técnico. Las explicamos desde cero:

Término	Qué significa
Cliente	El programa o dispositivo que pide cosas: tu navegador, la app del móvil, <code>psql</code> , una automatización
Servidor	Un ordenador (o un programa) que está siempre encendido y responde a las peticiones de otros
API	Una «ventanilla» por la que un programa pide o envía datos a otro, con reglas fijas sobre qué se puede pedir y cómo
VPS	Un servidor virtual alquilado: una parte de un ordenador de un proveedor que tú controlas como si fuera tuyo
Base de datos en la nube	Una base de datos que alquilas ya instalada, configurada y mantenida por un proveedor

Para entender «servidor», piensa en un restaurante. El cliente es el comensal que pide. El servidor es la cocina, siempre disponible, que prepara lo que se le pide. La API es la carta y el camarero: dicen qué se puede pedir y cómo se pide.

¿Para qué sirve?

Saber dónde vive cada pieza te permite entender cómo viaja un dato desde la pantalla hasta el disco, y dónde puede fallar. Cuando algo no funciona, lo primero que se hace es preguntar: ¿en qué pieza está el problema?

¿Por qué lo necesito?

Tu aplicación y tu base de datos casi nunca están en el mismo sitio. La web se ejecuta en el navegador de cada usuario; la base de datos, en otro ordenador. Entre ambas hay piezas intermedias. Entender este recorrido es la base para desplegar, proteger y diagnosticar tus proyectos.

¿Cómo funciona?

El recorrido típico de un dato:

1. El **cliente** (navegador o móvil) envía una petición a la **API**.
2. La API, que se ejecuta en un **servidor**, comprueba quién eres y qué puedes ver.
3. La API pregunta a la **base de datos**.
4. La base de datos responde a la API.

5. La API devuelve la respuesta al cliente, que la muestra en pantalla.

Normalmente el cliente **no** habla directamente con la base de datos: una API está en medio. Se explica por qué en la Parte 15.

Dónde vive el servidor puede variar: en un **VPS** que tú alquilas y administras (control total y también la responsabilidad de mantenerlo; Parte 8), o en un servicio de **base de datos en la nube**, donde el proveedor lo mantiene (Partes 9 y 10).

Para que un cliente encuentre al servidor necesita tres datos: **dónde está** (su dirección o nombre), **por qué puerta se entra** (el **puerto**, un número; PostgreSQL usa por defecto el 5432) y **con qué identidad** (usuario y, normalmente, contraseña). Se resumen en una **cadena de conexión**:

Parte de	Significado
<code>postgresql://postgres@localhost:5433/tienda</code>	
<code>postgresql://</code>	El protocolo: se va a hablar con PostgreSQL
<code>postgres</code>	El usuario
<code>@localhost</code>	La máquina donde está el servidor (<code>localhost</code> significa «este mismo ordenador»)
<code>:5433</code>	El puerto
<code>/tienda</code>	La base de datos a la que se conecta

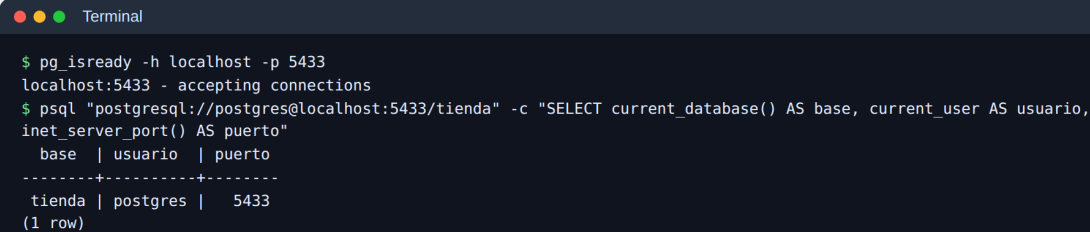
(Si el usuario tuviera contraseña, iría detrás del usuario: `usuario:contraseña@`. En este entorno de pruebas no la hay; nunca será así en producción.)

Paso a paso: seguir una petición real de cliente a base de datos

1. **Comprueba que el servidor responde** con `pg_isready`.
2. **Conéctate como cliente** con la cadena de conexión y pregunta qué base de datos, usuario y puerto estás usando.
3. **Mira el código de una API mínima**: un programa de Node.js que se conecta a la base de datos y ofrece la dirección `/clientes`.
4. **Arranca la API** pasándole la cadena de conexión por una variable de entorno.
5. **Haz de cliente**: pide `/clientes` con `curl` y comprueba que llega la información.
6. **Pide una dirección que no existe** y observa la respuesta 404.
7. **Para la API**.

Código y resultado: el servidor y la conexión

```
pg_isready -h localhost -p 5433
psql "postgresql://postgres@localhost:5433/tienda" -c "SELECT current_database()
AS base, current_user AS usuario, inet_server_port() AS puerto"
```



```
Terminal
$ pg_isready -h localhost -p 5433
localhost:5433 - accepting connections
$ psql "postgresql://postgres@localhost:5433/tienda" -c "SELECT current_database() AS base, current_user AS usuario,
inet_server_port() AS puerto"
  base | usuario | puerto
-----+-----+-----
 tienda | postgres |   5433
(1 row)
```

Figura 1.10-a. Captura real: comprobar que el servidor está disponible y conectarse con una cadena de conexión.

Qué ves en la imagen.

1. `pg_isready -h localhost -p 5433` pregunta al servidor si está listo: responde `accepting connections` («aceptando conexiones»).
2. `psql "postgresql://postgres@localhost:5433/tienda" -c "..."` se conecta con la cadena de conexión y ejecuta una consulta (`-c` significa «ejecuta esta orden y sal»). Devuelve la base de datos (`tienda`), el usuario (`postgres`) y el puerto (`5433`): coinciden con las partes de la cadena.

Código y resultado: una API mínima entre el cliente y la base de datos

Este es el código de la API, un archivo `server.js` de Node.js (un programa que ejecuta JavaScript fuera del navegador). Es una API mínima: solo contesta a la dirección `/clientes`.

```
const http = require('http');
const { Pool } = require('pg');

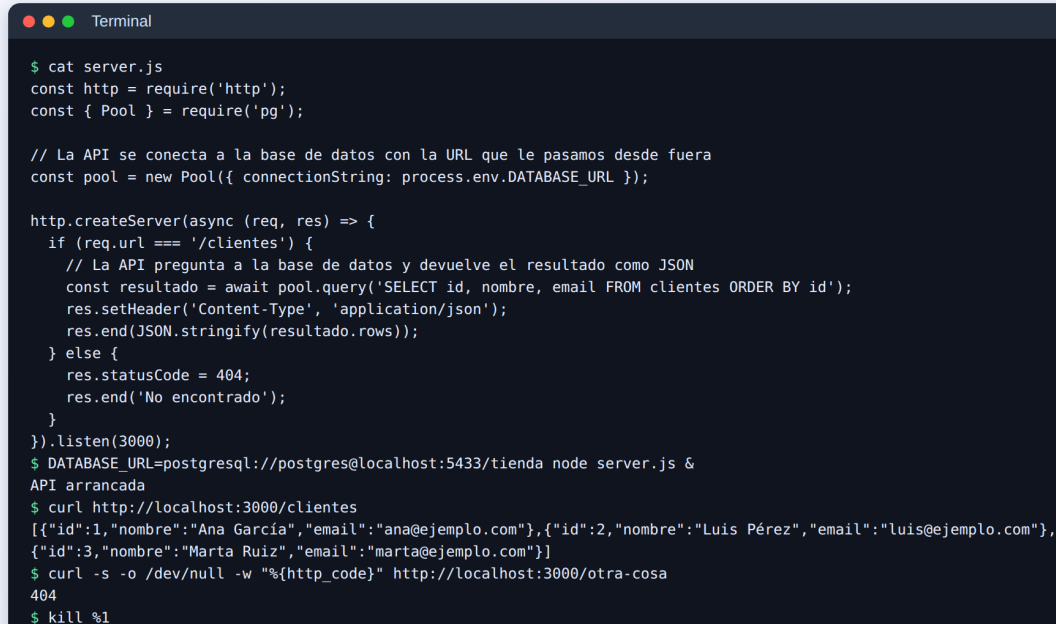
// La API se conecta a la base de datos con la URL que le pasamos desde fuera
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

http.createServer(async (req, res) => {
  if (req.url === '/clientes') {
    // La API pregunta a la base de datos y devuelve el resultado como JSON
    const resultado = await pool.query('SELECT id, nombre, email FROM clientes
ORDER BY id');
    res.setHeader('Content-Type', 'application/json');
    res.end(JSON.stringify(resultado.rows));
  } else {
    res.statusCode = 404;
    res.end('No encontrado');
  }
}).listen(3000);
```

Lectura sencilla:

- `require('pg')` carga la librería que sabe hablar con PostgreSQL. `Pool` es un grupo de conexiones reutilizables.
- `process.env.DATABASE_URL`: la cadena de conexión no está escrita en el código, se recibe del entorno (**variable de entorno**). Así las contraseñas nunca viajan dentro del código (Parte 17).
- `http.createServer(...)` crea el servidor web, y `.listen(3000)` lo pone a escuchar en el puerto 3000.
- Si la dirección pedida es `/clientes`, la API **pregunta a la base de datos** (`pool.query(...)`) y devuelve las filas como **JSON** (un formato de texto que casi todos los programas entienden). Para cualquier otra dirección responde con el código 404 («no encontrado»).

```
cat server.js
DATABASE_URL=postgresql://postgres@localhost:5433/tienda node server.js &
curl http://localhost:3000/clientes
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/otra-cosa
kill %1
```



```
Terminal
$ cat server.js
const http = require('http');
const { Pool } = require('pg');

// La API se conecta a la base de datos con la URL que le pasamos desde fuera
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

http.createServer(async (req, res) => {
  if (req.url === '/clientes') {
    // La API pregunta a la base de datos y devuelve el resultado como JSON
    const resultado = await pool.query('SELECT id, nombre, email FROM clientes ORDER BY id');
    res.setHeader('Content-Type', 'application/json');
    res.end(JSON.stringify(resultado.rows));
  } else {
    res.statusCode = 404;
    res.end('No encontrado');
  }
}).listen(3000);
$ DATABASE_URL=postgresql://postgres@localhost:5433/tienda node server.js &
API arrancada
$ curl http://localhost:3000/clientes
[{"id":1,"nombre":"Ana García","email":"ana@ejemplo.com"}, {"id":2,"nombre":"Luis Pérez","email":"luis@ejemplo.com"}, {"id":3,"nombre":"Marta Ruiz","email":"marta@ejemplo.com"}]
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/otra-cosa
404
$ kill %1
```

Figura 1.10-b. Captura real: el recorrido completo. El cliente (`curl`) pide a la API, la API pregunta a PostgreSQL y devuelve las filas.

Qué ves en la imagen.

1. `cat server.js` muestra el código de arriba.
2. `DATABASE_URL=... node server.js &` arranca la API pasándole la cadena de conexión (en la captura se ejecuta en segundo plano, que es lo que significa el `&` final).

3. `curl http://localhost:3000/clientes` hace de cliente: pide `/clientes` a la API. La respuesta es una lista JSON con los tres clientes: **son las filas de la tabla `clientes`**, que la API ha leído de PostgreSQL. Fíjate en que **no aparece el teléfono**: la consulta de la API solo pidió `id`, `nombre` y `email`.
4. Pedir otra dirección devuelve el código `404`.
5. `kill %1` detiene la API.

Este es el recorrido de los puntos 1-5 de «¿Cómo funciona?»: cliente → API → base de datos → API → cliente.

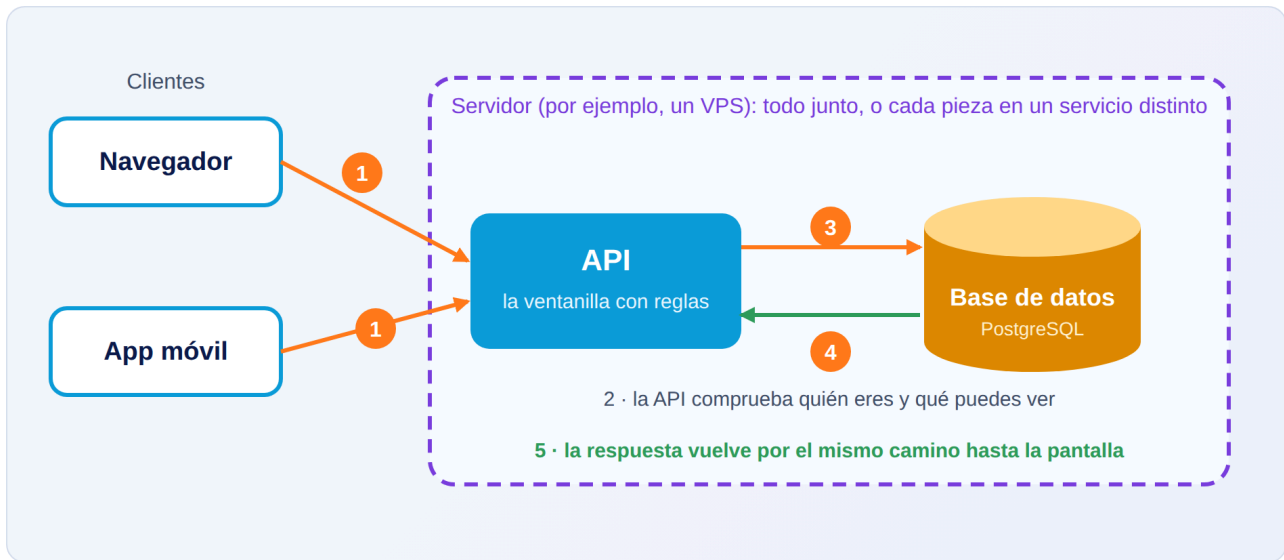


Figura 1.10-c. Ilustración: recorrido de una petición.

Qué ves en la imagen. A la izquierda, los clientes (navegador y móvil). Las flechas numeradas muestran la petición hacia la API (que se ejecuta en un servidor) y hacia la base de datos, y la respuesta que vuelve por el mismo camino. El recuadro punteado indica que la API y la base de datos pueden vivir en un mismo VPS o en servicios distintos.

Ejemplo sencillo

Abres una app y pulsas «Mis pedidos». Tu móvil (cliente) pide los pedidos a la API. La API pregunta a la base de datos. La base de datos devuelve las filas, la API las envía al móvil y el móvil las dibuja.

Ejemplo real

Una pequeña empresa alquila un VPS, instala en él PostgreSQL y su API, y los usa desde una web y una app móvil. Otra alquila solo la base de datos a un proveedor y pone la API en otro servicio. En ambos casos el recorrido es el mismo; cambia dónde vive cada pieza.

Profundizando

Por qué el cliente no habla directamente con la base de datos. Si la web o la app se conectaran directamente, la cadena de conexión (con la contraseña) tendría que estar dentro de la

aplicación, al alcance de cualquiera que la inspeccione. La API es la que guarda los secretos, comprueba quién eres y decide qué puedes ver (Parte 15).

Puertos habituales. PostgreSQL usa por defecto el 5432; las webs, el 80 (HTTP) y el 443 (HTTPS). En este entorno de pruebas el servidor usa el 5433 y la API el 3000.

localhost. Significa «este mismo ordenador». Cuando el servidor está en otro ordenador, en su lugar va su dirección o su nombre (por ejemplo, el de tu VPS, Parte 8).

Más piezas. Entre el cliente y la API pueden existir otras (balanceadores, cachés, proxies). Las veremos en las partes de despliegue.

Ejercicio

1. Ordena el recorrido de una petición: base de datos, navegador, API.
2. Clasifica como cliente, servidor o API: la app de tu móvil, el ordenador de un proveedor que siempre está encendido, la «ventanilla» por la que se piden los pedidos.
3. Interpreta esta cadena de conexión: `postgresql://ana@miservidor.com:5432/crm`.
¿Quién se conecta, dónde y a qué base de datos?
4. Explica con tus palabras la diferencia entre alquilar un VPS e instalar tú PostgreSQL, y alquilar una base de datos en la nube.

Solución

1. Navegador → API → base de datos; la respuesta vuelve por el mismo camino.
2. La app del móvil: cliente. El ordenador del proveedor: servidor. La ventanilla: API.
3. Se conecta el usuario `ana`, al servidor `miservidor.com`, por el puerto `5432`, a la base de datos `crm`.
4. En un VPS tú instalas, configuras, actualizas y proteges PostgreSQL: control total y más trabajo. En una base de datos en la nube, el proveedor se ocupa de buena parte del mantenimiento, a cambio de menos control y de pagar por el servicio (Parte 9).

Error habitual

Pensar que el navegador o la app hablan directamente con la base de datos, y escribir la contraseña de la base de datos dentro del código.

```
// Mal: la contraseña dentro del código
const pool = new Pool({ connectionString:
  'postgresql://postgres:miClave@servidor/tienda' });
// Bien: la cadena llega desde el entorno
const pool2 = new Pool({ connectionString: process.env.DATABASE_URL });
```

Buena práctica

Antes de montar cualquier proyecto, dibuja en un papel el recorrido de los datos: quién pide, quién responde, dónde está cada pieza. Y guarda las cadenas de conexión fuera del código.

Comprobación

Para uno de tus proyectos, escribe en una línea dónde vive cada pieza: cliente, API y base de datos. Si alguna no la sabes situar, es la primera que debes aclarar. Después comprueba con `pg_isready` que tu servidor responde.

Resumen de la Parte 1

- Los datos sueltos no dicen nada; con contexto (columnas, tipos, filas) se convierten en información.
- Las bases de datos existen para guardar muchos datos sin duplicados ni contradicciones, con reglas, permisos y uso simultáneo.
- El gestor es el programa; la base de datos son los datos. Un gestor contiene muchas bases de datos independientes y aplica los permisos.
- Una tabla tiene columnas (campos) y filas (registros); cada celda contiene un valor.
- Cada columna tiene un tipo de dato, y el tipo decide lo que puedes hacer con él.
- NULL significa «no hay valor»; se pregunta con `IS NULL`, nunca con `= NULL`.
- La clave primaria identifica filas; la clave extranjera enlaza tablas y protege la coherencia.
- Los índices aceleran las búsquedas, pero tienen un coste.
- Una consulta es una petición al gestor y devuelve una tabla de resultados.
- El cliente pide a una API, y la API consulta la base de datos con una cadena de conexión que no se escribe en el código.

Glosario de la Parte 1

Término	Significado
Dato	Valor suelto, sin contexto
Información	Datos con contexto que les da sentido
Base de datos	Conjunto organizado de datos relacionados
SGBD (gestor)	Programa que crea, guarda, protege y consulta bases de datos
Tabla	Datos de un mismo tipo de cosa, en filas y columnas
Columna / campo	Una característica que tienen todos los elementos de la tabla
Fila / registro	Un elemento concreto de la tabla
Valor	Dato que hay en el cruce de una fila y una columna
Tipo de dato	Clase de valor que admite una columna
NULL	Ausencia de valor
Clave primaria (PK)	Columna que identifica cada fila de forma única
Clave extranjera (FK)	Columna que guarda la clave primaria de otra tabla
Índice	Estructura auxiliar que acelera las búsquedas
Consulta	Petición que se hace al gestor
SQL	Lenguaje con el que se escriben las consultas
<code>psql</code>	Programa de terminal para hablar con PostgreSQL
Cliente	Quien pide datos (navegador, app, automatización)
Servidor	Ordenador o programa que responde a peticiones
API	Ventanilla con reglas por la que un programa pide datos a otro
VPS	Servidor virtual alquilado
Puerto	Número de la «puerta» por la que se entra a un servidor
Cadena de conexión	Texto que indica usuario, servidor, puerto y base de datos
Variable de entorno	Valor que recibe un programa desde fuera de su código
JSON	Formato de texto para intercambiar datos entre programas

Mini examen de la Parte 1

Responde sin mirar las respuestas; después compáralas.

1. ¿Qué diferencia hay entre un dato y una información? Pon un ejemplo.
2. ¿Qué diferencia hay entre PostgreSQL y una base de datos llamada `tienda`?
3. En la tabla CLIENTES, ¿qué es una fila y qué es una columna?
4. ¿Qué tipo de dato usarías para un precio y por qué?
5. ¿Qué significa NULL y cómo se pregunta por él?
6. ¿Para qué sirve la clave primaria?
7. En PEDIDOS, ¿qué columna es la clave extranjera y qué protege?
8. ¿Qué ventaja tiene un índice y qué coste?
9. ¿Qué devuelve una consulta?
10. Ordena el recorrido: base de datos, API, móvil. ¿Por qué no se conecta el móvil directamente a la base de datos?

Respuestas

1. Un dato es un valor sin contexto («34»); la información tiene contexto («Ana tiene 34 años»).
2. PostgreSQL es el gestor (el programa); `tienda` es una base de datos que vive dentro de él.
3. Una fila es un registro (un cliente concreto); una columna es un campo (una característica, como `email`).
4. Decimal exacto (`numeric`), porque necesita céntimos y se va a sumar y comparar.
5. Significa «no hay valor»; se pregunta con `IS NULL` , no con `= NULL` .
6. Para identificar cada fila de forma única, sin repeticiones ni vacíos.
7. `cliente_id` ; protege que no haya pedidos de clientes inexistentes ni se borren clientes con pedidos.
8. Acelera las búsquedas; cuesta espacio y tiempo al escribir, porque hay que mantenerlo actualizado.
9. Una tabla de resultados con cero, una o muchas filas.
10. Móvil → API → base de datos (y la respuesta vuelve por el mismo camino). Porque la contraseña de la base de datos tendría que estar dentro de la app, al alcance de cualquiera.

Si has acertado 8 o más, estás listo para la Parte 2. Si no, repasa los puntos de las preguntas falladas.